



Rapport de fin d'Étude

Préparer en vue de l'obtention du :
Diplôme licence en Science de l'Informatique
parcours Génie Logiciel et Systèmes d'Informations

Conception et Réalisation d'un système qui gère les tâches dans un service après vente :

BEST Management System

Réalisé par : Jlassi Moaataz

Organisme d'accueil :



Bright Electronic Solutions Technology (BEST)

Encadrant pédagogique : M. JANDOUBI Slim

Encadrant professionnel : M. B. HAMMOUDA Bilel

Années universitaires : 2024 – 2025

Table des matières

1	Introduction générale	1
2	Présentation du cadre général	3
2.1	Introduction	3
2.2	Présentation de l'organisme d'accueil BEST	3
2.2.1	Aperçu général	3
2.2.2	Missions et services	3
2.2.3	Valeurs et engagements	4
2.3	Organigramme de la société	4
2.3.1	Compétences humaines	4
2.4	Les technologies du projet	5
2.4.1	Frontend	5
2.4.2	Backend	5
2.5	Conclusion	5
3	Analyse et spécification des besoins	6
3.1	Introduction	6
3.2	Description de l'existant	6
3.2.1	Problématiques identifiées	6
3.2.2	Conséquences	7
3.3	Besoins fonctionnels détaillés	7
3.3.1	Authentification et gestion des utilisateurs	7
3.3.2	Gestion des réparations	7
3.3.3	Gestion des pièces de stock	8
3.3.4	Notifications internes	8
3.3.5	Pièces et prix	9
3.3.6	Gestion des agences et marques	9
3.3.7	Ventes d'accessoires	9
3.4	Besoins non fonctionnels détaillés	9
3.4.1	Performance	9
3.4.2	Sécurité	9
3.4.3	Scalabilité	10
3.4.4	Fiabilité	10
3.4.5	Maintenabilité	10
3.4.6	Compatibilité	10

3.4.7	Ergonomie	10
3.5	Solutions similaires et analyse comparative	10
3.5.1	Solutions existantes	10
3.5.2	Avantages de notre solution	11
3.5.3	Limites du système	11
3.6	Priorisation des besoins	11
3.7	Architecture de la solution	12
3.8	Réponse aux besoins fonctionnels	13
3.9	Réponse aux besoins non fonctionnels	13
3.10	Conclusion	13
4	Modélisation de l'application	14
4.1	Introduction	14
4.2	Diagramme de cas d'utilisation	14
4.3	Diagramme de séquence	17
4.3.1	Principe général de l'authentification	17
4.4	Diagramme de classes	18
4.4.1	Entités principales	18
4.4.2	Relations principales	20
4.5	Diagramme d'activité	22
4.6	Diagramme d'états	24
4.7	Diagramme de déploiement	26
4.8	Conclusion	27
5	Conception technique détaillée	28
5.1	Introduction	28
5.2	Architecture logicielle	28
5.2.1	Architecture du backend (NestJS)	28
5.2.2	Architecture du frontend (React + Redux)	29
5.2.3	Instance Axios centralisée	29
5.3	Structure de la base de données	29
5.3.1	Schéma relationnel	29
5.3.2	Index et optimisation	33
5.4	Data Transfer Objects (DTOs)	33
5.4.1	Validation des entrées	33
5.4.2	Exemple de DTO – Création d'utilisateur	33
5.4.3	Exemple de DTO – Création de réparation	34
5.5	Flux d'authentification et sécurité	35
5.5.1	Processus d'authentification détaillé	35
5.5.2	Accès protégé et renouvellement	35
5.5.3	Sécurité avancée	35
5.6	Endpoints API	36
5.6.1	Module Authentification	36
5.6.2	Module Utilisateurs	36
5.6.3	Module Réparations	36

5.6.4	Module Stock	37
5.6.5	Module Transferts	37
5.6.6	Module Administration	38
5.7	Conclusion	38
6	Réalisation	39
6.1	Introduction	39
6.2	Technologies utilisées	39
6.2.1	Frontend : React 19 + Vite 6 + TypeScript	39
6.2.2	Design UI : Material UI v7	40
6.2.3	Backend : NestJS	41
6.2.4	Base de données : PostgreSQL	41
6.3	Implémentation	41
6.3.1	Organisation du backend (NestJS)	41
6.3.2	Organisation du frontend (React)	42
6.3.3	Gestion des rôles et navigation dynamique	43
6.3.4	Seed de la base de données	43
6.4	Scénarios d'utilisation	44
6.4.1	Authentification	44
6.4.2	Tableau de bord	44
6.4.3	Gestion des agences	44
6.4.4	Gestion des utilisateurs	45
6.4.5	Gestion des marques et modèles	46
6.4.6	Réception d'un produit	48
6.4.7	Affectation au technicien	49
6.4.8	Réparation par le technicien	50
6.4.9	Contrôle qualité	51
6.4.10	Récupération par le client	51
6.5	Conclusion	52
7	Tests et validation	53
7.1	Introduction	53
7.2	Stratégie de test	53
7.3	Tests unitaires frontend	53
7.3.1	Outil utilisé : Vitest + Testing Library	53
7.3.2	Résultats	53
7.3.3	Exemple de test	54
7.4	Tests d'intégration backend	54
7.4.1	Outil utilisé : Jest + Supertest	54
7.4.2	Résultats	54
7.4.3	Exemple de test d'intégration	54
7.5	Tests E2E Playwright	55
7.5.1	Configuration	55
7.5.2	Scénarios de test	55
7.5.3	Exemple de test E2E	56

7.6	Bilan des tests	57
7.7	Conclusion	57
8	Conclusion générale	58
A	Annexe : Détails des entités et relations	60
A.1	Les entités du système	60
A.2	Les méthodes par entité	61
A.3	Relations entre les classes	62
A.3.1	Relations du module réparation	62
A.3.2	Relations du module client et appareil	63
A.3.3	Relations du module stock	63
A.3.4	Relations du module utilisateur et traçabilité	64

Rapport Projet BEST, réalisé par: Moataz

Liste des tableaux

3.1	Classement priorisé des fonctionnalités	11
4.1	Description des cas d'utilisation	15
4.2	Description des interactions du diagramme de séquence	18
4.3	Description des entités principales du diagramme de classes	21
4.4	Description des étapes du diagramme d'activité	24
4.5	Description des états d'une réparation	25
4.6	Description des composants du déploiement	27
5.1	Tables de référence du système	30
5.2	Tables de gestion du système	31
5.3	Endpoints du module d'authentification	36
5.4	Endpoints du module utilisateurs	36
5.5	Endpoints du module réparations	36
5.6	Endpoints du module stock	37
5.7	Endpoints du module transferts	37
5.8	Endpoints du module administration	38
6.1	Tableau comparatif des frameworks frontend	39
6.2	Tableau comparatif des frameworks UI	40
6.3	Tableau comparatif des frameworks backend	41
7.1	Répartition des tests E2E par module	55
7.2	Bilan de la couverture des tests	57
A.1	Description détaillée des entités	60
A.2	Méthodes disponibles par entité	61

Table des figures

2.1	Organigramme de la société BEST	4
4.1	Diagramme de cas d'utilisation global	15
4.2	Diagramme de séquence global du système	17
4.3	Diagramme de classes complet du système	21
4.4	Diagramme d'activité – Flux de réparation complet	23
4.5	Diagramme d'états d'une réparation	25
4.6	Diagramme de déploiement	26
6.1	React + Vite + TypeScript	39
6.2	Material UI	40
6.3	NestJS	41
6.4	PostgreSQL	41
6.5	Page d'authentification	44
6.6	Tableau de bord avec quatre rôles	44
6.7	Tableau de bord avec un seul rôle	44
6.8	Navigation pour ajouter une agence	45
6.9	Formulaire d'ajout d'une agence	45
6.10	Navigation pour ajouter un utilisateur	46
6.11	Formulaire d'ajout d'un utilisateur avec sélection de rôles	46
6.12	Navigation pour ajouter une marque	47
6.13	Formulaire d'ajout d'une marque	47
6.14	Navigation pour ajouter un modèle	47
6.15	Formulaire d'ajout d'un modèle	48
6.16	Réception d'un produit – navigation et formulaire	48
6.17	PDF généré pour la réparation	49
6.18	Envoi du dossier vers l'affectation	49
6.19	Validation des produits en attente d'affectation	50
6.20	Affectation d'un produit à un technicien	50
6.21	Le technicien accepte les affectations	50
6.22	Saisie des détails de réparation	51
6.23	Contrôle qualité	51
6.24	Récupération des produits par le client	52

Chapitre 1

Introduction générale

Le développement technologique est aujourd'hui un élément essentiel de notre époque moderne. Il s'accompagne d'évolutions continues dans de nombreux domaines, qu'ils soient industriels, économiques, touristiques ou liés aux services. Ces secteurs s'appuient fortement sur les systèmes d'information, qui permettent d'organiser et de coordonner les activités afin d'améliorer la rapidité et la qualité des services.

Dans le domaine des services, en particulier lors du contact direct avec les clients, la rapidité et la précision des informations sont cruciales pour éviter tout problème. Pour atteindre cet objectif, il est nécessaire de mettre en place un système capable de gérer le travail de manière efficace, tout en réduisant l'effort demandé aux utilisateurs.

Le secteur du Service Après-Vente (SAV) dans le domaine de la téléphonie mobile est confronté à des défis spécifiques : gestion d'un volume élevé de réparations, suivi des pièces détachées, traçabilité des interventions, coordination entre les équipes (réception, technique, stock, administration) et communication avec les clients. Un système de gestion adapté est indispensable pour relever ces défis et garantir un service de qualité.

C'est dans ce contexte que s'inscrit le projet **BEST Management System**, une application web complète destinée à la gestion des tâches d'un service après-vente spécialisé dans la réparation de smartphones et accessoires. Ce projet a été développé au sein de l'entreprise **Bright Electronic Solutions Technology (BEST)**, spécialisée dans le SAV pour les marques internationales.

Afin de répondre à ces enjeux, nous avons conçu et réalisé un système de gestion moderne, combinant efficacité, ergonomie et évolutivité. Ce rapport présente l'ensemble des étapes suivies pour atteindre cet objectif :

- **Chapitre 1 : Présentation du cadre général** – Ce chapitre introduit l'organisme d'accueil, son fonctionnement, son organigramme, ainsi que les outils et technologies utilisés.

- **Chapitre 2 : Analyse et spécification des besoins** – Les besoins fonctionnels et non fonctionnels du système sont analysés, ainsi que les solutions existantes et leur priorisation.
- **Chapitre 3 : Modélisation de l'application** – Ce chapitre décrit la conception du système à travers six diagrammes UML (cas d'utilisation, classes, séquence, activité, états, déploiement).
- **Chapitre 4 : Conception technique détaillée** – L'architecture logicielle, le schéma de la base de données, les DTOs, la sécurité et les endpoints API sont présentés en détail.
- **Chapitre 5 : Réalisation** – Les technologies, l'implémentation et les scénarios d'utilisation sont décrits avec des captures d'écran.
- **Chapitre 6 : Tests et validation** – Les stratégies de test (unitaires, intégration, E2E Playwright) et leurs résultats sont présentés.

Chapitre 2

Présentation du cadre général

2.1 Introduction

Ce chapitre présente le cadre général du projet. Il décrit l'organisme d'accueil, son rôle dans le domaine du service après-vente, son organigramme et sa structure interne. Il présente également les outils et technologies nécessaires à la réalisation du projet.

2.2 Présentation de l'organisme d'accueil BEST

2.2.1 Aperçu général

BEST (Bright Electronic Solutions Technology) est une société privée tunisienne opérant dans le secteur des services, spécialisée dans l'après-vente et la réparation d'appareils électroniques. La société collabore avec des marques internationales telles qu'Infinix et Tecno, pour lesquelles elle assure le service après-vente officiel en Tunisie.

2.2.2 Missions et services

BEST constitue l'intermédiaire officiel entre les marques partenaires et les clients finaux pendant toute la durée du partenariat. Ses missions principales incluent :

- La réparation des appareils (smartphones, tablettes, accessoires) sous garantie et hors garantie.
- La gestion des pièces détachées et du stock.
- Le service client et la communication avec les utilisateurs.
- La maintenance préventive et corrective des équipements.
- La traçabilité complète des opérations de réparation.

2.2.3 Valeurs et engagements

BEST met à disposition la main-d'œuvre nécessaire pour réaliser les réparations avec professionnalisme et dans les délais les plus courts. La société s'engage à :

- Fournir un service de qualité avec des techniciens qualifiés.
- Garantir la satisfaction client à chaque étape du processus.
- Assurer une transparence totale dans les opérations.
- Respecter les normes et standards des marques partenaires.

2.3 Organigramme de la société

L'organigramme ci-dessous présente l'architecture hiérarchique de BEST :

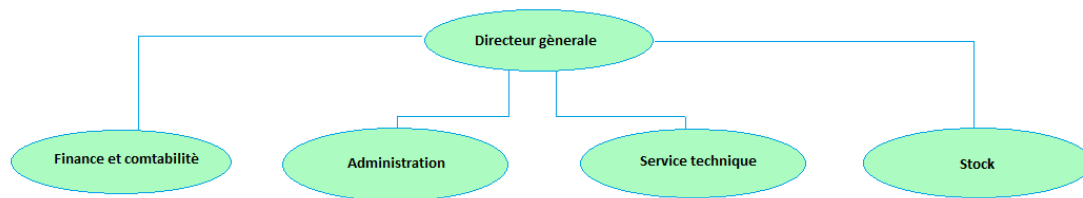


FIGURE 2.1 – Organigramme de la société BEST

La structure organisationnelle de BEST comprend les entités suivantes :

- **Direction générale** : Supervision globale et stratégie.
- **Service réception** : Accueil des clients, réception des appareils et remise des produits réparés.
- **Service technique** : Diagnostic, réparation et contrôle qualité des appareils.
- **Gestion des stocks** : Gestion des pièces détachées, approvisionnements et transferts.
- **Service administratif** : Facturation, gestion des utilisateurs et administration.

2.3.1 Compétences humaines

Les compétences humaines nécessaires comprennent :

- Capacité à poser un diagnostic précis sur différents types de pannes.
- Réactivité et performance face à une charge de travail élevée.
- Maîtrise des différentes marques et modèles d'appareils.

- Compétences en communication avec la clientèle.
- Gestion du temps et des priorités.

2.4 Les technologies du projet

Notre solution repose sur un ensemble de technologies modernes :

2.4.1 Frontend

- **React 19** : Bibliothèque JavaScript pour la construction d'interfaces utilisateur.
- **Vite 6** : Outil de build rapide et moderne.
- **TypeScript** : Superset JavaScript avec typage statique.
- **Material UI v7** : Bibliothèque de composants design.
- **Redux Toolkit** : Gestion d'état global.
- **React Router 7** : Routage côté client.

2.4.2 Backend

- **NestJS** : Framework Node.js modulaire et structuré.
- **TypeORM** : ORM pour la gestion de la base de données.
- **PostgreSQL** : SGBD relationnel avancé.
- **JWT** : Authentification par tokens.
- **Helmet** : Sécurisation des en-têtes HTTP.
- **Throttler** : Rate limiting pour l'API.

2.5 Conclusion

Ce chapitre a permis de présenter l'organisme d'accueil BEST, son organigramme, ses outils et les technologies employées. Ces éléments constituent la base de compréhension du cadre de travail dans lequel s'inscrit notre solution de gestion.

Chapitre 3

Analyse et spécification des besoins

3.1 Introduction

Dans le cadre de mon projet web, il est essentiel de définir les besoins afin de garantir une solution optimisée et conforme. Ces besoins sont classés en deux catégories : besoins fonctionnels et besoins non fonctionnels. L'analyse des besoins constitue une étape fondamentale qui permet de comprendre les attentes des utilisateurs et de définir le périmètre du projet.

3.2 Description de l'existant

Avant la mise en place de notre solution, le fonctionnement du SAV présentait plusieurs lacunes importantes :

3.2.1 Problématiques identifiées

- **Absence d'inventaire du stock** : Impossible de connaître la quantité de pièces disponibles en réserve.
- **Impossibilité de vérifier l'état du stock** : Aucune visibilité sur l'état (bonne/défectueuse) des pièces en magasin.
- **Manque de visibilité comptable** : Impossible de consulter la comptabilité des pièces globalement ou par modèle.
- **Compte unique partagé** : Tout le personnel travaillait avec un seul compte, sans traçabilité des actions.
- **Absence de mise à jour des informations** : Les données n'étaient pas actualisées régulièrement.
- **Traçabilité inexistante** : Impossible de suivre le cycle de vie d'un produit (entrée, étapes de réparation, sortie).
- **Consultation difficile des prix** : Le prix final et la disponibilité d'une pièce n'étaient pas accessibles rapidement.

- **Transfert manuel** : Les transferts de produits ou pièces entre agences se faisaient sans suivi numérique.

3.2.2 Conséquences

Ces lacunes entraînaient :

- Des pertes de temps pour rechercher l'information.
- Des erreurs de communication entre les équipes.
- Une mauvaise expérience client.
- Une difficulté à prendre des décisions basées sur des données fiables.
- Un manque de professionnalisme dans la gestion quotidienne.

3.3 Besoins fonctionnels détaillés

3.3.1 Authentification et gestion des utilisateurs

Chaque utilisateur doit se connecter avec son identifiant et son mot de passe sécurisé pour accéder au système. Le système gère également :

- La création et la gestion des comptes utilisateurs.
- L'attribution de rôles multiples à un même utilisateur.
- La gestion des sessions via tokens JWT avec Access Token et Refresh Token.
- La déconnexion sécurisée.
- La récupération de mot de passe.

Dans cet environnement de travail, un utilisateur peut occuper plusieurs rôles (par exemple, réceptionniste et gestionnaire de stock). Contrairement à des solutions similaires où il doit changer de compte pour accomplir ses tâches, ici un seul compte suffit pour réaliser toutes les opérations. Le menu de navigation est dynamique et s'adapte aux rôles de l'utilisateur.

3.3.2 Gestion des réparations

Le système doit permettre :

- La création d'un dossier de réparation complet (client + appareil + détails).
- L'enregistrement des problèmes identifiés lors du diagnostic.
- La vérification de l'historique du même appareil.
- L'affectation des réparations aux techniciens.
- Le suivi des étapes de réparation (8 états).
- La génération de fiches PDF pour le suivi.

- La gestion des garanties (sous garantie / hors garantie).
- L'ajout de pièces nécessaires (partsNeed) à chaque réparation.
- L'approbation des pièces utilisées par le gestionnaire de stock.

3.3.3 Gestion des pièces de stock

État de stock

Gestion des pièces (bonnes ou défectueuses) dans chaque agence, avec :

- Consultation de l'état global de toutes les pièces.
- Filtrage par état, référence, emplacement.
- Export des résultats vers Excel.
- Alertes de stock faible (seuil configurable par l'administrateur).

Remplissage de stock

Ajout de pièces au stock avec :

- Saisie rapide des références, quantités et emplacements.
- Import par fichier Excel avec format prédéfini.
- Gestion des numéros de série pour les pièces traçables.
- Association des pièces aux cases de stockage (Bin) et références.

Transfert des pièces

Le système permet de :

- Transférer des pièces vers d'autres agences.
- Suivre l'état d'avancement d'un transfert (envoyé, reçu, annulé).
- Notifier les destinataires du transfert.
- Historiser tous les mouvements de stock.

3.3.4 Notifications internes

Le système de notification inclut :

- Alertes de rupture de stock envoyées chaque semaine.
- Notifications pour les responsables de stock de chaque agence.
- Notifications à l'administrateur pour l'état global des stocks.
- Notifications en temps réel via WebSockets pour les approbations.

3.3.5 Pièces et prix

Le système permet à tout utilisateur de :

- Consulter la disponibilité et le prix TTC d'une pièce.
- Le prix inclut la main-d'œuvre et la TVA.
- Visualiser les prix par modèle et niveau de réparation.
- Accéder aux références compatibles.

3.3.6 Gestion des agences et marques

- Création et modification d'agences.
- Gestion des marques et modèles d'appareils.
- Association des modèles aux marques et types.

3.3.7 Ventes d'accessoires

- Vente d'accessoires (chargeurs, écouteurs, câbles USB).
- Gestion des stocks d'accessoires.
- Génération de factures de vente.

3.4 Besoins non fonctionnels détaillés

3.4.1 Performance

- Temps de réponse API inférieur à 200ms pour les requêtes standards.
- Pagination des résultats pour les listes volumineuses.
- Optimisation des requêtes SQL avec des index.
- Mécanismes de cache côté serveur.
- Support de la connexion simultanée de plusieurs utilisateurs.

3.4.2 Sécurité

- Authentification sécurisée via cookies HTTP-only.
- Chiffrement des mots de passe avec bcrypt (12 rounds de salage).
- Protection CSRF via SameSite cookie attribute.
- Rate limiting (60 requêtes/minute) avec NestJS Throttler.
- Protection des en-têtes HTTP avec Helmet.
- Validation stricte des entrées utilisateur (ValidationPipe + whitelist).

3.4.3 Scalabilité

- Architecture modulaire permettant l'ajout de nouvelles fonctionnalités.
- Possibilité d'ajouter des agences sans modifier l'architecture.
- Support de l'ajout de nouveaux rôles utilisateur.

3.4.4 Fiabilité

- Gestion robuste des erreurs avec des messages explicites.
- Validation des données côté client et serveur.
- Tests unitaires (70+ suites, 678 tests frontend).
- Tests d'intégration (78 suites, 801 tests backend).
- Tests E2E Playwright (89 tests).

3.4.5 Maintenabilité

- Code bien structuré avec TypeScript.
- Architecture modulaire (modules NestJS).
- Documentation des API avec commentaires.
- Respect des principes SOLID.
- Utilisation d'un style de code cohérent.

3.4.6 Compatibilité

- Fonctionnement fluide sur Chrome, Firefox, Edge.
- Design responsive (smartphones, tablettes et ordinateurs).

3.4.7 Ergonomie

- Interface claire et intuitive avec Material UI.
- Mode sombre / clair.
- Tableaux dynamiques avec filtres et recherche.
- Navigation réduite basée sur les rôles.

3.5 Solutions similaires et analyse comparative

3.5.1 Solutions existantes

Il existe d'autres solutions de gestion de SAV, mais elles présentent des limitations :

- **Informations dispersées** : Les données sont réparties dans différents emplacements, rendant le processus de vérification long et fastidieux.
- **Absence de fusion de rôles** : L'utilisateur doit changer de compte pour chaque rôle.
- **Charge quantitative élevée** : Beaucoup d'étapes manuelles non nécessaires.
- **Disponibilité des accessoires** : Impossible de vérifier rapidement la disponibilité d'un accessoire.
- **Prix incomplets** : Les solutions similaires n'incluent pas la TVA ni la main-d'œuvre.
- **Facturation simpliste** : Pas de gestion des remises ni des frais supplémentaires (logistique, etc.).
- **Temps de réponse lent** : Délais importants pour obtenir une réponse.

3.5.2 Avantages de notre solution

- Accès rapide et simple à des informations précises et centralisées.
- Interface intuitive et confortable, adaptée aux utilisateurs non techniques.
- Optimisation de la rapidité du travail, même avec un volume élevé de tâches.
- Informations exploitables avec un minimum d'efforts.
- Gestion multi-rôles avec un seul compte.
- Calcul automatique des prix incluant TVA et main-d'œuvre.

3.5.3 Limites du système

- Disponible dans une seule langue (français).
- Pas de liste de tâches à faire intégrée.
- Système relationnel nécessitant une démarche spécifique.
- Non intégré aux systèmes des marques partenaires.

3.6 Priorisation des besoins

La priorisation des besoins consiste à classer les différentes fonctionnalités selon leur importance et leur urgence :

TABLE 3.1 – Classement priorisé des fonctionnalités

N°	Description	Priorité et détails
----	-------------	---------------------

1	Authentification et sécurité	Gestion des comptes, connexion par cookies HTTP-only, gestion des rôles multiples. Essentiel.
2	Gestion des agences	Création et modification des agences. Indispensable pour le déploiement.
3	Gestion des utilisateurs	Gestion des utilisateurs (arrivées, départs). Essentiel pour les RH.
4	Gestion marque/modèle	Ajout et modification des modèles réparés. Nécessaire pour la réception.
5	Gestion des réparations	Cycle complet (création, diagnostic, pièces, approbation, sortie). Cœur du système.
6	Gestion des stocks	Suivi entrées/sorties, emplacements, approbation. Critique pour le SAV.
7	Transfert de pièces/produits	Transfert entre agences avec suivi d'état. Important pour la flexibilité.
8	Tableaux dynamiques	Affichage avec filtres, recherche, actions sur lignes. Amélioration UX majeure.
9	Rapports et exports	Génération PDF et Excel pour état stock, réparations. Moyen.
10	Notifications internes	Alertes temps réel (stock faible, approbations). Amélioration communication.
11	Personnalisation UI	Mode sombre, personnalisation des vues. Secondaire.
12	Améliorations ergonomiques	Drag-and-drop, raccourcis clavier, animations. Optionnel.

3.7 Architecture de la solution

La solution proposée est basée sur une architecture client-serveur en trois couches :

- **Frontend** : Application React + Vite + TypeScript, offrant une interface ergonomique et dynamique.
- **Design UI** : Material UI pour un design clair, moderne, responsive et accessible.
- **Gestion d'état** : Redux Toolkit pour centraliser et gérer les données (authentification, réparations, stock, notifications).
- **Backend API** : NestJS (Node.js + TypeScript) pour une API REST structurée et modulaire.
- **Base de données** : PostgreSQL pour un stockage relationnel fiable et performant.

3.8 Réponse aux besoins fonctionnels

- **Authentification** : Système sécurisé par cookies HTTP-only + JWT, gestion multi-rôles avec menu dynamique.
- **Gestion des réparations** : Module complet avec suivi d'état (8 étapes), pièces nécessaires, PDF, historique.
- **Gestion des stocks** : Entités StockPart, Reference, Bin avec alertes, transferts et historique.
- **Notifications internes** : Alertes temps réel via WebSockets, notifications planifiées.
- **Pièces et prix** : Consultation centralisée avec TVA et main-d'œuvre.

3.9 Réponse aux besoins non fonctionnels

- **Performance** : Optimisation SQL (index), pagination, cache.
- **Sécurité** : bcrypt (12 rounds), cookies HTTP-only, Helmet, Throttler (60 req/min), ValidationPipe whitelist.
- **Scalabilité** : Architecture modulaire NestJS, conteneurisation Docker possible.
- **Fiabilité** : Tests unitaires (678), intégration (801), E2E (89).
- **Maintenabilité** : TypeScript, modules NestJS, principes SOLID.
- **Compatibilité** : Responsive design, multi-navigateurs.

3.10 Conclusion

Ce chapitre a permis d'identifier et de clarifier les besoins fonctionnels et non fonctionnels du projet. L'analyse détaillée a mis en évidence les exigences essentielles pour garantir la qualité, la performance et la fiabilité du système. Cette étape constitue une base solide pour la conception et la réalisation de l'application.

Chapitre 4

Modélisation de l'application

4.1 Introduction

Dans le cadre du développement d'un projet logiciel, il est essentiel de définir la structure, le comportement et les interactions du système avant de commencer la phase de réalisation. Pour ce faire, nous utilisons le langage de modélisation UML (Unified Modeling Language), dont l'objectif est de fournir une représentation graphique et normalisée des différents aspects du système.

Pour ce projet, nous avons réalisé six diagrammes UML complémentaires :

- **Diagramme de cas d'utilisation** : Représente les interactions entre les acteurs et le système.
- **Diagramme de classes** : Décrit la structure statique du système (entités, attributs, relations).
- **Diagramme de séquence** : Montre les interactions temporelles entre les objets.
- **Diagramme d'activité** : Modélise les flux de travail et les processus métier.
- **Diagramme d'états** : Décrit les états possibles d'une réparation et leurs transitions.
- **Diagramme de déploiement** : Présente l'architecture physique du système.

4.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation global ci-dessous regroupe l'ensemble des interactions entre les acteurs et le système. Il présente les cinq acteurs principaux (Administrateur, Coordinateur, Technicien, Réceptionniste, Gestionnaire de stock) et leurs cas d'utilisation respectifs, couvrant l'intégralité des fonctionnalités du BEST Management System.

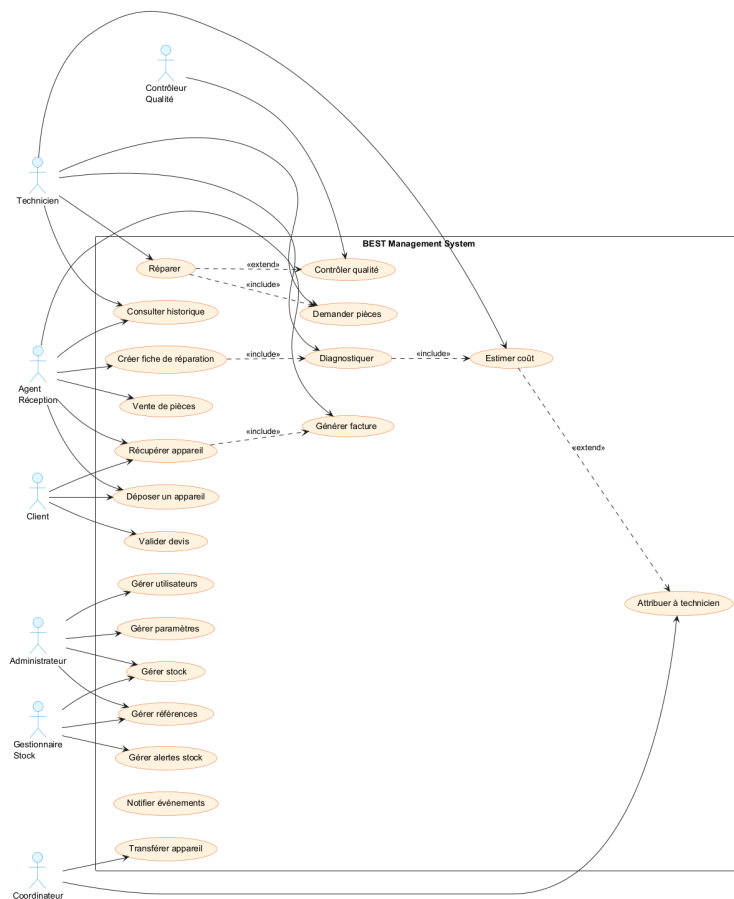


FIGURE 4.1 – Diagramme de cas d'utilisation global

TABLE 4.1 – Description des cas d'utilisation

Acteur	Cas d'utilisation	Description
Réceptionniste	Gérer les clients	Création et recherche de clients
Réceptionniste	Gérer les appareils	Réception et identification des appareils
Réceptionniste	Créer et suivre les réparations	Enregistrement et suivi des réparations
Réceptionniste	Générer les fiches PDF	Production des fiches de suivi réparation
Réceptionniste	Gérer les factures	Création et impression des factures
Réceptionniste	Vente d'accessoires	Vente d'accessoires aux clients
Réceptionniste	Remettre les produits réparés	Restitution au client avec rapport

Acteur	Cas d'utilisation	Description
Technicien	Consulter les affectations	Visualisation des tâches assignées
Technicien	Accepter les réparations	Prise en charge des affectations
Technicien	Remplir les détails réparation	Diagnostic, actions et pièces utilisées
Technicien	Ajouter des fichiers	Photos et documents de réparation
Technicien	Gérer la garantie	Détermination du statut garantie
Technicien	Envoyer pour contrôle qualité	Soumission au CQ
Gestionnaire stock	Remplir le stock	Entrées de pièces détachées
Gestionnaire stock	Consulter l'état du stock	Visualisation des niveaux de stock
Gestionnaire stock	Transférer les pièces	Transfert vers d'autres agences
Gestionnaire stock	Gérer les références et prix	Configuration des pièces
Gestionnaire stock	Approuver les pièces	Validation des pièces utilisées
Coordinateur	Affecter les produits	Assignment aux techniciens
Coordinateur	Contrôler la qualité	Vérification des réparations
Coordinateur	Transférer les produits	Transfert entre agences
Coordinateur	Suivre l'avancement	Supervision des techniciens
Administrateur	Gérer les agences	CRUD des agences
Administrateur	Gérer les utilisateurs	CRUD et attribution des rôles
Administrateur	Gérer les marques/modèles	Configuration du catalogue
Administrateur	Consulter les statistiques	Tableaux de bord globaux
Administrateur	Configurer le système	Seuils d'alerte, TVA, etc.

4.3 Diagramme de séquence

4.3.1 Principe général de l'authentification

Tous les rôles doivent s'authentifier auprès du système avant de commencer leurs tâches. Le processus est le suivant :

1. L'utilisateur saisit son identifiant et mot de passe dans le formulaire de login.
2. Le frontend envoie une requête POST /auth/login au AuthController.
3. Le AuthService vérifie les identifiants dans la base de données.
4. Si les identifiants sont valides, un Access Token (15 min) et un Refresh Token (7 jours) sont générés.
5. Le Refresh Token est stocké dans un cookie HTTP-only.
6. Les informations utilisateur sont stockées dans le store Redux.
7. L'utilisateur est redirigé vers le tableau de bord.

Le diagramme de séquence ci-dessous présente le flux global des interactions entre les acteurs et le système pour le scénario principal de réparation :

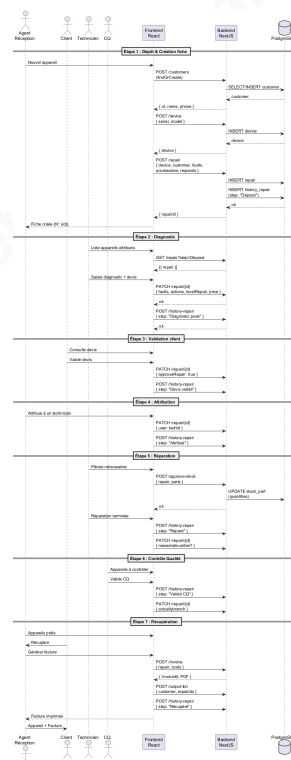


FIGURE 4.2 – Diagramme de séquence global du système

TABLE 4.2 – Description des interactions du diagramme de séquence

Acteur	Interaction	Détails
Réceptionniste	Réception produit	Enregistrement client, appareil et création de la réparation
Réceptionniste	Génération fiche PDF	Création du document de suivi réparation
Réceptionniste	Envoi à affectation	Transmission du dossier au coordinateur
Coordinateur	Affectation technicien	Validation et assignation à un technicien
Technicien	Acceptation affectation	Prise en charge de la réparation
Technicien	Diagnostic et réparation	Saisie des détails techniques et pièces
Technicien	Envoi contrôle qualité	Soumission de la réparation au CQ
Coordinateur	Validation CQ	Approbation ou refus de la réparation
Réceptionniste	Remise au client	Restitution de l'appareil réparé avec rapport
Réceptionniste	Génération liste sortie	Clôture du processus

4.4 Diagramme de classes

Le diagramme de classes présente l'ensemble des entités du système et leurs relations. Notre base de données comprend plus de 30 entités interconnectées.

4.4.1 Entités principales

Le système est organisé autour des entités suivantes :

Entités de gestion des utilisateurs

- **Company** : Entreprise (nom, logo, coordonnées bancaires, seuil d'alerte stock).
- **Branch** : Agence (nom, adresse, téléphone, email).
- **User** : Utilisateur (nom, login, mot de passe, rôles, statut, agence de rattachement).
- **Tracability** : Traçabilité des actions utilisateurs.

Entités de gestion des réparations

- **Customer** : Client (nom, téléphone, distributeur associé).
- **Distributeur** : Distributeur/revendeur (nom, contact, localisation).
- **Device** : Appareil (numéro de série, date d'achat).
- **Model** : Modèle d'appareil (nom, image).
- **Brand** : Marque (nom, logo, statut).
- **TypeModel** : Type de modèle (smartphone, écouteurs, etc.).
- **Repair** : Réparation (garantie, état, pièces nécessaires, fichiers, remarques).
- **HistoryRepair** : Historique des étapes de réparation.
- **RepairAction** : Action de réparation effectuée.
- **LevelRepair** : Niveau de réparation (avec prix de main-d'œuvre).
- **ListFault** : Liste des problèmes.
- **CustomerRequest** : Demande du client.
- **NotesCustomer** : Notes pour le client.
- **ExpertiseReason** : Raison d'expertise (hors garantie).
- **Accessory** : Accessoire reçu avec l'appareil.

Entités de gestion des stocks

- **AllPart** : Liste complète des pièces (description).
- **PartsPrice** : Prix des pièces par modèle.
- **Reference** : Référence de pièce (code matière, description).
- **Bin** : Case de stockage (nom, type).
- **StockPart** : Pièce en stock (numéro de série, remarque).
- **ApproveStock** : Approbation de pièce (type, date, état).
- **HistoryStockPart** : Historique de mouvement de pièce.
- **Transfert** : Transfert de pièces entre agences.
- **StockAlert** : Alerte de stock.

Entités de facturation et ventes

- **Invoice** : Facture (méthode de paiement, date, état, prix total).
- **OtherCost** : Autres frais (logistique, etc.).
- **Sale** : Vente d'accessoire (état, prix total, date).
- **OutputList** : Liste de sortie des produits réparés.

4.4.2 Relations principales

Le diagramme de classes complet est présenté ci-dessous. Les relations principales incluent :

- **Repair – Accessory** (ManyToMany) : Association des accessoires reçus avec chaque réparation.
- **Repair – ListFault** (ManyToMany) : Problèmes identifiés lors du diagnostic.
- **Repair – CustomerRequest** (ManyToMany) : Demandes spécifiques du client.
- **Repair – NotesCustomer** (ManyToMany) : Messages destinés au client.
- **Repair – ExpertiseReason** (ManyToMany) : Raisons de mise hors garantie.
- **Repair – RepairAction** (ManyToOne) : Action de réparation appliquée.
- **Repair – OutputList** (OneToMany) : Liste de sortie contenant les réparations.
- **Repair – Customer** (ManyToOne) : Client propriétaire de l'appareil.
- **Repair – Device** (ManyToOne) : Appareil réparé.
- **Repair – ApproveStock** (OneToMany) : Pièces approuvées pour la réparation.
- **Repair – Transfert** (ManyToMany) : Transferts de réparations entre agences.
- **Repair – HistoryRepair** (OneToMany) : Historique des étapes.
- **Repair – Invoice** (OneToOne) : Facture associée (hors garantie).
- **Repair – User** (ManyToOne) : Technicien affecté.

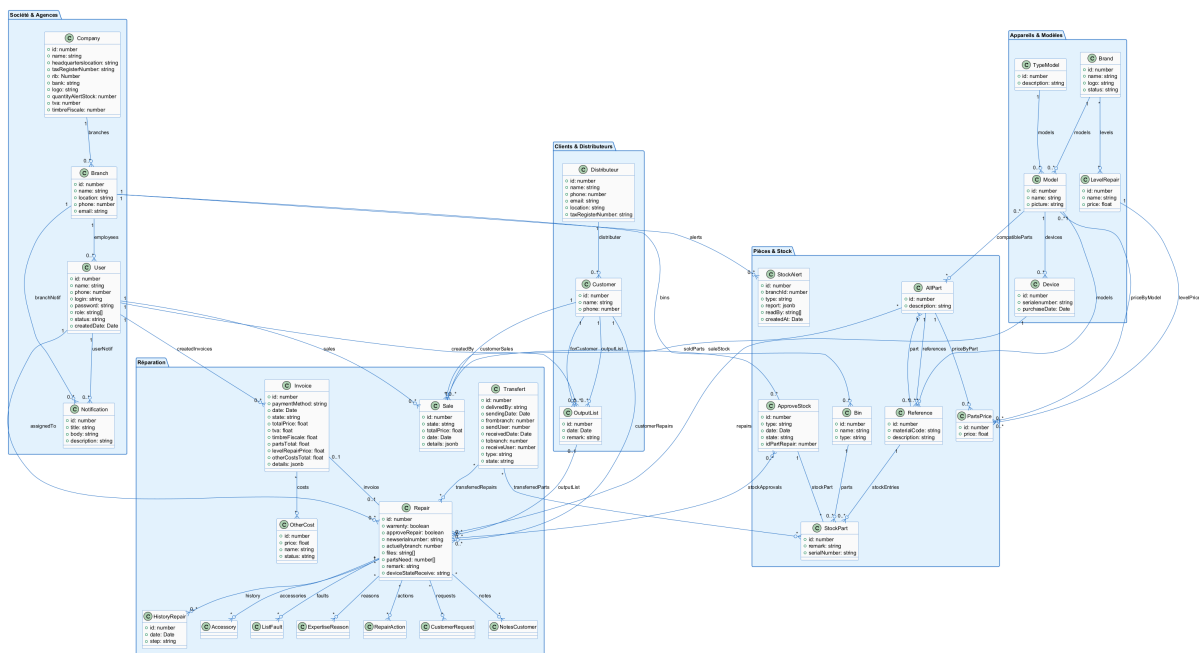


FIGURE 4.3 – Diagramme de classes complet du système

TABLE 4.3 – Description des entités principales du diagramme de classes

Module	Entité	Description
Gestion utilisateurs	Company	Entreprise (nom, logo, coordonnées bancaires)
Gestion utilisateurs	Branch	Agence (nom, adresse, téléphone)
Gestion utilisateurs	User	Utilisateur (nom, login, rôles, statut, agence)
Gestion utilisateurs	Tracability	Traçabilité des actions utilisateurs
Réparations	Customer	Client (nom, téléphone, distributeur)
Réparations	Device	Appareil (numéro de série, date d'achat)
Réparations	Repair	Réparation (garantie, état, pièces, fichiers)
Réparations	Model	Modèle d'appareil (nom, image, marque)
Réparations	Brand	Marque (nom, logo, statut)
Réparations	RepairAction	Action de réparation effectuée

Module	Entité	Description
Réparations	LevelRepair	Niveau de réparation (prix MO)
Réparations	HistoryRepair	Historique des étapes de réparation
Stocks	AllPart	Liste complète des pièces
Stocks	Reference	Référence de pièce (code matière)
Stocks	StockPart	Pièce en stock (numéro de série)
Stocks	Bin	Case de stockage (nom, type)
Stocks	Transfert	Transfert de pièces entre agences
Facturation	Invoice	Facture (paiement, date, état, prix)
Facturation	Sale	Vente d'accessoire (état, prix, date)
Facturation	OtherCost	Autres frais (logistique)

4.5 Diagramme d'activité

Le diagramme d'activité ci-dessous modélise le flux de travail complet depuis la réception d'un appareil jusqu'à sa restitution au client. Ce diagramme montre la séquence d'étapes, les points de décision et les rôles impliqués à chaque étape du processus.

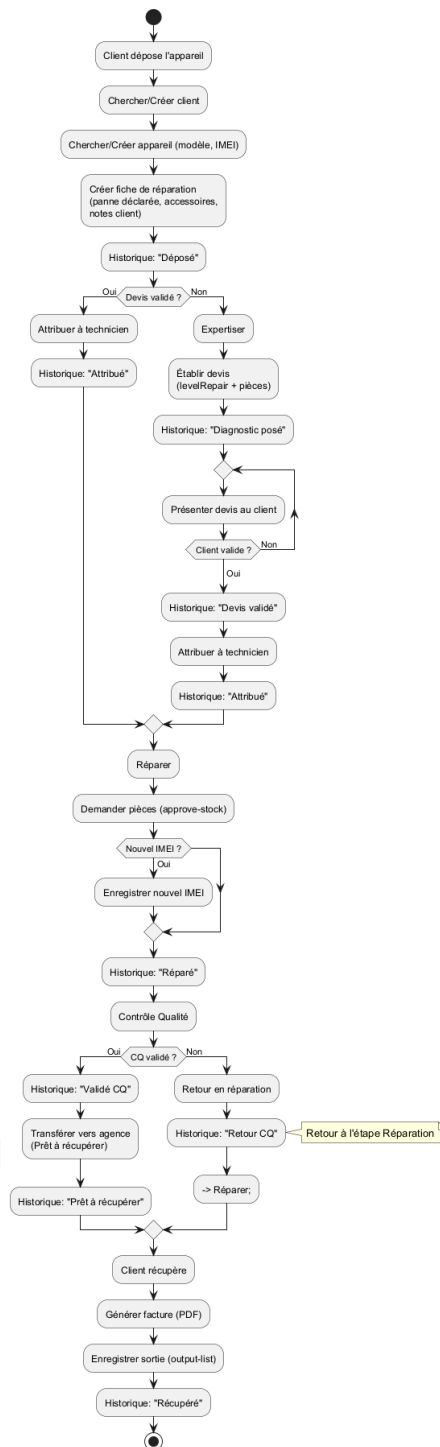


FIGURE 4.4 – Diagramme d'activité – Flux de réparation complet

TABLE 4.4 – Description des étapes du diagramme d'activité

N°	Acteur	Description
1	Réceptionniste	Enregistrement du client, de l'appareil et création de la réparation
2	Réceptionniste	Génération de la fiche de suivi PDF
3	Réceptionniste	Envoi du dossier au coordinateur pour affectation
4	Coordinateur	Affectation de la réparation à un technicien
5	Technicien	Acceptation de l'affectation par le technicien
6	Technicien	Diagnostic, réparation et saisie des détails techniques
7	Coordinateur	Contrôle qualité de la réparation effectuée
8	Coordinateur	Approbation (suite) ou refus (retour technicien)
9	Réceptionniste	Remise de l'appareil réparé au client
10	Réceptionniste	Génération de la liste de sortie et clôture

4.6 Diagramme d'états

Le diagramme d'états ci-dessous présente les huit états possibles d'une réparation dans le système, ainsi que les transitions entre ces états :

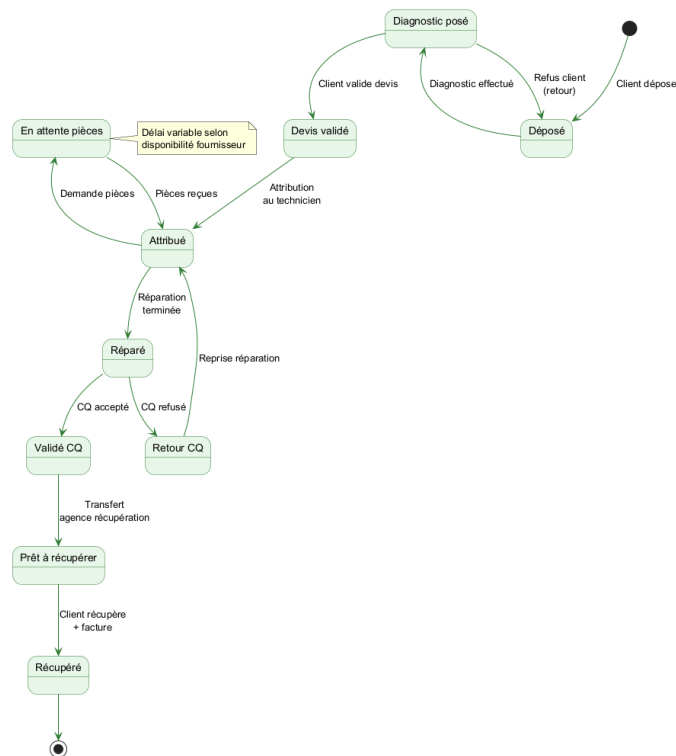


FIGURE 4.5 – Diagramme d'états d'une réparation

TABLE 4.5 – Description des états d'une réparation

Code	Libellé	Description
En Recept	En réception	Appareil enregistré par le réceptionniste
Envoye A affecter	Envoyé à affecter	Dossier transmis au coordinateur
Envoye Au Technicien	Envoyé au technicien	Affectation réalisée par le coordinateur
En Reparation	En réparation	Technicien en cours de réparation
Envoye En CQ	Envoyé en CQ	Réparation soumise au contrôle qualité
Refuse	Refusé (recyclé)	CQ refusé ; retour au technicien
En Recupération	En récupération	Approuvée, prête à être remise
Cloture	Clôturé	Client a récupéré l'appareil

4.7 Diagramme de déploiement

Le diagramme de déploiement ci-dessous présente l'architecture physique du système, montrant la répartition des composants logiciels sur les nœuds matériels :

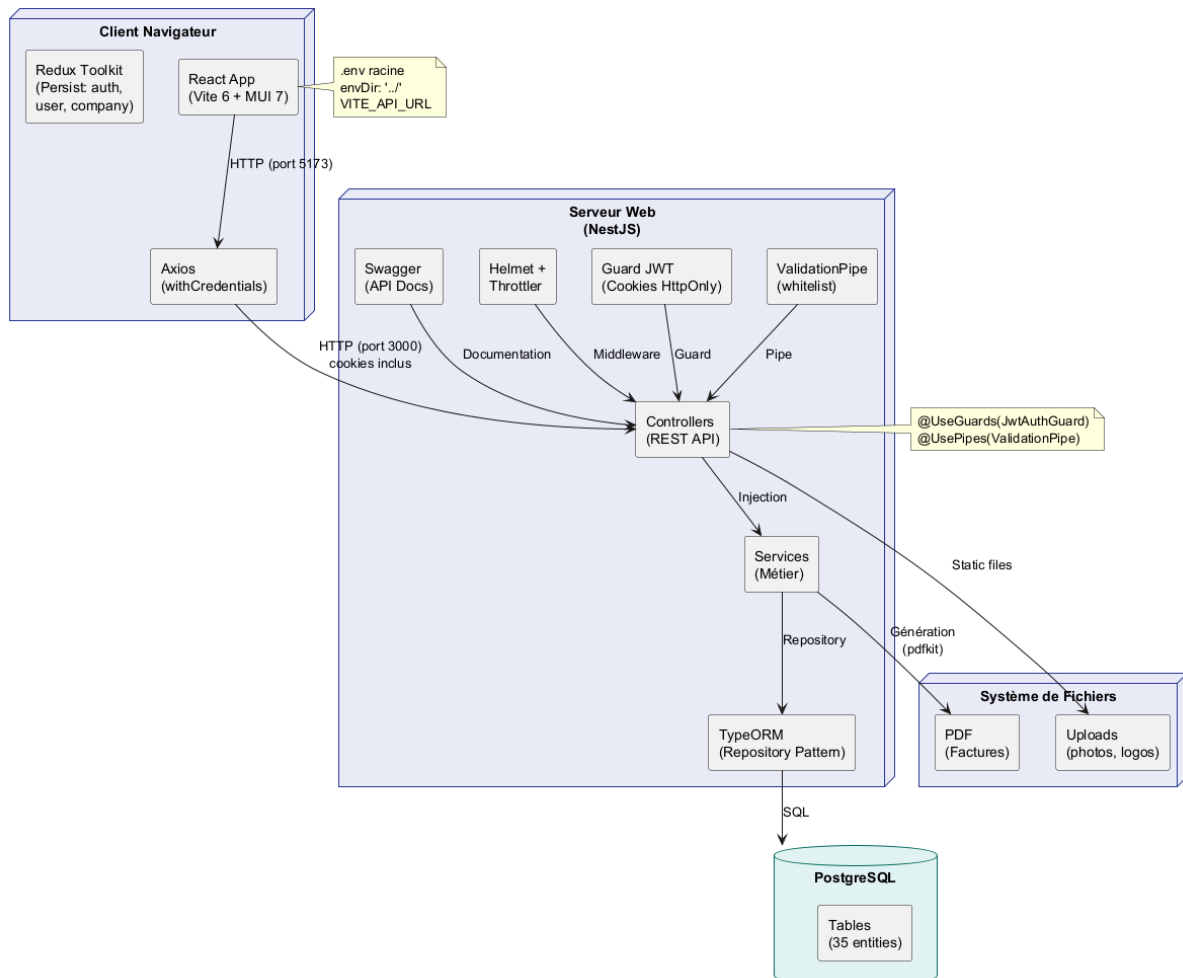


FIGURE 4.6 – Diagramme de déploiement

L'architecture de déploiement comprend trois tiers :

1. **Tier présentation (Client)** : Navigateur web (Chrome, Firefox, Edge) sur poste de travail.
2. **Tier serveur (Backend)** : Serveur Node.js hébergeant l'API NestJS avec Web-Sockets.
3. **Tier données** : Serveur PostgreSQL stockant toutes les données relationnelles.

TABLE 4.6 – Description des composants du déploiement

Tier	Composant	Technologies
Présentation	Client web	Navigateur (Chrome, Firefox, Edge)
Présentation	Application React	Vite + Redux + Material UI
Serveur	API REST	NestJS + TypeScript
Serveur	WebSockets	Notifications temps réel
Serveur	Authentification	JWT + Refresh Token
Données	Base de données	PostgreSQL
Données	Protocole	TCP/IP

4.8 Conclusion

Ce chapitre a permis de modéliser l'application à travers six diagrammes UML complémentaires. Cette modélisation a fourni une vision complète et structurée du système, couvrant à la fois les aspects statiques (classes), dynamiques (séquences, activités, états) et physiques (déploiement). Cette étape constitue une base solide pour la phase de réalisation.

Chapitre 5

Conception technique détaillée

5.1 Introduction

Ce chapitre présente la conception technique détaillée du système BEST Management System. Il couvre l'architecture logicielle, la structure de la base de données, les DTOs, le flux de sécurité et les endpoints API.

5.2 Architecture logicielle

5.2.1 Architecture du backend (NestJS)

Le backend est organisé en modules indépendants suivant le pattern modulaire de NestJS :

- **AuthModule** : Authentification, login, logout, refresh token.
- **UsersModule** : Gestion CRUD des utilisateurs.
- **BranchModule** : Gestion des agences.
- **BrandModule** : Gestion des marques.
- **ModelModule** : Gestion des modèles d'appareils.
- **DeviceModule** : Gestion des appareils.
- **CustomerModule** : Gestion des clients.
- **RepairModule** : Gestion des réparations (cœur du système).
- **StockModule** : Gestion des pièces de stock.
- **TransfertModule** : Gestion des transferts.
- **InvoiceModule** : Gestion des factures.
- **SaleModule** : Gestion des ventes.
- **OutputListModule** : Gestion des listes de sortie.
- **NotificationModule** : Gestion des notifications.

- **ApproveStockModule** : Approbation des pièces.
- **Plus de 25 modules au total.**

Chaque module contient :

- **Controller** : Expose les endpoints REST.
- **Service** : Contient la logique métier.
- **Entity** : Modèle TypeORM lié à la base de données.
- **Module** : Configuration du module NestJS.
- **DTO** : Data Transfer Objects pour la validation.

5.2.2 Architecture du frontend (React + Redux)

Le frontend est organisé par fonctionnalités :

- **pages/** : Pages principales (Login, Dashboard, Stock, Réparations, etc.) – 45+ pages.
- **components/** : Composants réutilisables (Tableaux dynamiques, formulaires, modales).
- **store/** : Gestion d'état Redux Toolkit (slices auth, user, company, repairs, stock, etc.).
- **services/** : Services API (instance Axios centralisée).
- **assets/** : Ressources statiques (logos, images).

5.2.3 Instance Axios centralisée

Tous les appels API passent par une instance Axios unique créée dans `services/api.ts`.

- `baseUrl` : Configurée via `VITE_API_URL` (fallback `http://localhost:3000`).
- `withCredentials: true` : Pour les cookies HTTP-only.
- Intercepteur de réponse : Gestion centralisée des erreurs 401 (refresh automatique du token).

5.3 Structure de la base de données

5.3.1 Schéma relationnel

La base de données `BEST_Managment_System` comprend plus de 30 tables reliées entre elles :

Tables de référence

TABLE 5.1 – Tables de référence du système

Table	Colonnes principales	Description
company	id, name, headquartersLocation, taxRegisterNumber, rib, bank, logo, quantityAlertStock	Informations de l'entreprise
branch	id, name, location, phone, email	Agence
user	id, name, phone, password, createDate, status, login, role	Utilisateur
brand	id, name, logo, status	Marque d'appareil
type_model	id, description	Type de modèle
model	id, name, picture	Modèle d'appareil
all_part	id, description	Liste complète des pièces
level_repair	id, name, price	Niveau de réparation
repair_action	id, name	Action de réparation
list_fault	id, name	Problème possible
customer_request	id, name	Demande client
notes_customer	id, name	Note pour le client
expertise_reason	id, name	Raison d'expertise
accessory	id, name	Accessoire
distributeur	id, name, phone, email, location, taxRegisterNumber	Distributeur
legislation	id, name	Législation

other_cost	id, price, name, status	Autres frais
------------	-------------------------	--------------

Tables de gestion

TABLE 5.2 – Tables de gestion du système

Table	Colonnes principales	Description
customer	id, name, phone, distributorId	Client
device	id, serialnumber, purchaseDate, modelId	Appareil
repair	id, warrenty, approveRepair, newserialnumber, actually-branch, files, partsNeed, remark, deviceStateReceive, deviceId, customerId, userId, repairActionId	Réparation
history_repair	id, date, step, repairId	Historique réparation
tracability	id, historyRepairId, historyStockPartId, userId	Traçabilité
reference	id, materialCode, description, modelId, allPartId	Référence pièce
bin	id, name, type, branchId	Case de stockage

stock_part	id, remark, serialnumber, binId, referenceld	Pièce en stock
approve_stock	id, type, date, state, idPartRepair, repairId, saleId, stockPartId	Approbation stock
history_stock	partId, date, step, stockPartId	Historique stock
transfert	id, deliveredBy, sendingDate, fromBranchId, sendUserId, receivedDate, toBranchId, receiveUserId, type, state, remark, typePart	Transfert
parts_price	id, price, modelId, allPartId, levelRepairId	Prix pièce
invoice	id, paymentMethod, date, state, totalPrice, repairId	Facture
output_list	id, date, remark, customerId, userId	Liste de sortie
sale	id, state, totalPrice, date, userId	Vente
stock_alert	id, branchId, referenceld	Alerte stock
notification	id, message, type, description, userId, branchId	Notification

5.3.2 Index et optimisation

Les indexes suivants ont été créés pour optimiser les performances :

- Index sur `repair.deviceId` : Pour les recherches d'historique par appareil.
- Index sur `repair.userId` : Pour les recherches par technicien.
- Index sur `stock_part.binId` : Pour les recherches par emplacement.
- Index sur `stock_part.referenceId` : Pour les recherches par référence.
- Index sur `transfert.fromBranchId` et `toBranchId` : Pour les transferts.

5.4 Data Transfer Objects (DTOs)

5.4.1 Validation des entrées

Tous les endpoints utilisent des DTOs avec les décorateurs `class-validator` pour valider les entrées. Le `ValidationPipe` global est configuré avec :

- `transform: true` : Transformation automatique des types.
- `whitelist: true` : Suppression des propriétés non décorées.
- `forbidNonWhitelisted: false` : Permet les propriétés optionnelles.

5.4.2 Exemple de DTO – Création d'utilisateur

Listing 5.1 – DTO de création d'utilisateur

```
export class CreateUserDto {  
  @IsString()  
  @IsNotEmpty()  
  name: string;  
  
  @IsString()  
  @IsNotEmpty()  
  login: string;  
  
  @IsString()  
  @IsNotEmpty()  
  phone: string;  
  
  @IsString()  
  @IsNotEmpty()  
  password: string;  
  
  @IsString({ each: true })  
  @IsArray()
```



```

    role: string[];

    @IsOptional()
    @IsNumber()
    branch?: number;

    @IsDateString()
    @IsOptional()
    createDate?: string;

    @IsString()
    @IsOptional()
    status?: string;
}

```

5.4.3 Exemple de DTO – Création de réparation

Listing 5.2 – DTO de création de réparation

```

export class CreateRepairDto {
    @IsNumber()
    @IsNotEmpty()
    customer: number;

    @IsNumber()
    @IsNotEmpty()
    device: number;

    @IsOptional()
    @IsNumber()
    levelRepair?: number;

    @IsOptional()
    @IsString()
    repairAction?: string;

    @IsOptional()
    partsNeed?: any;

    @IsOptional()
    remark?: string;

    @IsOptional()
    warranty?: boolean;
}

```

5.5 Flux d'authentification et sécurité

5.5.1 Processus d'authentification détaillé

Le processus d'authentification se déroule en plusieurs étapes :

1. **Login** : L'utilisateur saisit login + mot de passe dans le formulaire React.
2. **Envoi au backend** : Le frontend envoie une requête POST à `/auth/login`.
3. **Vérification** : NestJS vérifie l'existence de l'utilisateur en base.
4. **Validation du mot de passe** : `bcrypt` compare le hash stocké avec le mot de passe fourni.
5. **Génération JWT** : Deux tokens sont créés :
 - **Access Token** : Durée de vie courte (15 minutes), transmis dans l'en-tête Authorization.
 - **Refresh Token** : Durée de vie longue (7 jours), stocké dans un cookie HTTP-only.
6. **Stockage Redux** : Les informations utilisateur (nom, rôles, agence) sont stockées dans le store Redux.
7. **Redirection** : L'utilisateur est redirigé vers le tableau de bord.

5.5.2 Accès protégé et renouvellement

- Chaque requête API inclut l'Access Token dans l'en-tête Authorization : Bearer.
- Un guard JWT vérifie la validité du token avant d'autoriser l'accès.
- Lorsque l'Access Token expire, l'intercepteur Axios détecte l'erreur 401.
- Le Refresh Token (cookie HTTP-only) est utilisé pour obtenir un nouvel Access Token.
- Si le Refresh Token est également expiré, l'utilisateur est redirigé vers la page de login.

5.5.3 Sécurité avancée

- **Helmet** : Protection des en-têtes HTTP (XSS, content-type sniffing, etc.) avec configuration `crossOriginResourcePolicy`: `'cross-origin'` pour permettre les images servies depuis le backend.
- **Throttler** : Rate limiting à 60 requêtes par minute pour prévenir les attaques par force brute.

- **CORS** : Configuration restreinte à l'origine du frontend avec `credentials: true`.
- **ValidationPipe** : Whitelist pour éviter l'injection de propriétés non autorisées.
- **Hashage bcrypt** : 12 rounds de salage pour les mots de passe.

5.6 Endpoints API

5.6.1 Module Authentification

TABLE 5.3 – Endpoints du module d'authentification

Méthode	Endpoint	Description
POST	/auth/login	Authentification utilisateur
POST	/auth/logout	Déconnexion
POST	/auth/refresh	Rafraîchissement du token
GET	/auth/profile	Profil de l'utilisateur connecté

5.6.2 Module Utilisateurs

TABLE 5.4 – Endpoints du module utilisateurs

Méthode	Endpoint	Description
GET	/users	Liste des utilisateurs (paginée)
GET	/users/ :id	Détails d'un utilisateur
POST	/users	Création d'un utilisateur
PATCH	/users/ :id	Modification d'un utilisateur
DELETE	/users/ :id	Suppression d'un utilisateur
GET	/users/by-branch/ :id	Utilisateurs par agence
GET	/users/assignable	Utilisateurs disponibles pour affectation

5.6.3 Module Réparations

TABLE 5.5 – Endpoints du module réparations

Méthode	Endpoint	Description
GET	/repair	Liste des réparations (paginée)
GET	/repair/ :id	Détails d'une réparation

POST	/repair	Création d'une réparation
PATCH	/repair/ :id	Mise à jour d'une réparation
GET	/repair/by-device/ :id	Réparations par appareil
GET	/repair/by-branch/ :id	Réparations par agence
GET	/repair/by-user/ :id	Réparations par utilisateur
GET	/repair/by-step/ :step	Réparations par étape
PATCH	/repair/ :id/approve	Approbation d'une réparation

5.6.4 Module Stock

TABLE 5.6 – Endpoints du module stock

Méthode	Endpoint	Description
GET	/stock-part	Liste des pièces en stock
POST	/stock-part	Ajout d'une pièce
GET	/stock-part/ :id	Détails d'une pièce
GET	/stock-part/by-bin/ :id	Pièces par case de stockage
GET	/stock-part/filter	Filtrage des pièces
GET	/stock-part/state	État global du stock
POST	/stock-part/import	Import par fichier Excel
GET	/reference	Liste des références
POST	/reference	Création d'une référence
GET	/bin	Liste des cases
POST	/bin	Création d'une case

5.6.5 Module Transferts

TABLE 5.7 – Endpoints du module transferts

Méthode	Endpoint	Description
GET	/transfert	Liste des transferts
POST	/transfert	Création d'un transfert
PATCH	/transfert/ :id	Mise à jour d'un transfert
GET	/transfert/from-branch/ :id	Transferts depuis une agence

GET	/transfert/to-branch/ :id	Transferts vers une agence
-----	---------------------------	----------------------------

5.6.6 Module Administration

TABLE 5.8 – Endpoints du module administration

Méthode	Endpoint	Description
GET/POST	/branch	Gestion des agences
GET/POST	/brand	Gestion des marques
GET/POST	/model	Gestion des modèles
GET/POST	/distributeur	Gestion des distributeurs
GET/POST	/all-parts	Gestion des pièces
GET/POST	/repair-action	Gestion des actions de réparation
GET/POST	/level-repair	Gestion des niveaux de réparation
GET/POST	/list-fault	Gestion des problèmes
GET/POST	/customer-request	Gestion des demandes client
GET/POST	/notes-customer	Gestion des notes client
GET/POST	/expertise-reasons	Gestion des raisons d'expertise
GET/POST	/legislation	Gestion des législations
GET/POST	/other-cost	Gestion des autres frais
GET/POST	/accessory	Gestion des accessoires

5.7 Conclusion

Ce chapitre a présenté la conception technique détaillée du système, couvrant l'architecture modulaire, le schéma de base de données (30+ entités), les DTOs avec validation, le flux d'authentification sécurisé et l'ensemble des endpoints API. Cette conception constitue le plan d'exécution pour la phase de réalisation.

Chapitre 6

Réalisation

6.1 Introduction

Ce chapitre est consacré à la réalisation concrète de l'application. Il présente les technologies utilisées, l'implémentation des modules, ainsi que les scénarios d'utilisation illustrés par des captures d'écran.

6.2 Technologies utilisées

6.2.1 Frontend : React 19 + Vite 6 + TypeScript

Le choix de ReactJS combiné à Vite et TypeScript se justifie par sa simplicité d'apprentissage, ses performances élevées et la richesse de son écosystème.



FIGURE 6.1 – React + Vite + TypeScript

TABLE 6.1 – Tableau comparatif des frameworks frontend

Critères	React (choisi)	+	Vite	Angular	Vue.js
----------	-------------------	---	------	---------	--------

Courbe d'apprentissage	Facile, grande communauté	Complexe, nécessite TypeScript avancé	Simple, adapté aux petits projets
Performance	Très rapide (Vite HMR)	Bonne mais compilation lourde	Bonne, moins optimisée que Vite
Écosystème	Très riche, nombreuses librairies	Complet et structuré mais rigide	Moins riche, communauté active
Flexibilité	Hautement flexible	Structure stricte	Flexible mais moins standardisé
Adoption industrielle	L'un des plus utilisés au monde	Utilisé dans grandes entreprises	Startups et projets moyens

6.2.2 Design UI : Material UI v7

Material UI offre un design moderne basé sur Material Design, une bibliothèque riche de composants réutilisables et une intégration parfaite avec React.



FIGURE 6.2 – Material UI

TABLE 6.2 – Tableau comparatif des frameworks UI

Critères	MUI (choisi)	Bootstrap	TailwindCSS
Design	Moderne, Material Design	Classique, peu personnalisable	Très flexible mais config lourde
Composants prêts	Très riche (tableaux, formulaires)	Large choix mais moins adapté	Pas de véritables composants
Personnalisation	Thèmes faciles à configurer	Possible mais lourde	Très personnalisable
Intégration React	Conçu nativement pour React	Compatible mais pas pensé React	Très bien intégré

6.2.3 Backend : NestJS

NestJS offre une structure modulaire, une compatibilité native avec TypeScript et un écosystème riche (WebSockets, JWT, Swagger).



FIGURE 6.3 – NestJS

TABLE 6.3 – Tableau comparatif des frameworks backend

Critères	NestJS (choisi)	Express.js	Django
Langage	TypeScript (typage fort)	JavaScript (flexible)	Python (hors écosystème JS)
Architecture	Modulaire et orientée services	Non structuré, manuelle	Intégrée mais rigide
Performance	Performant (Node.js async)	Très performant mais peu d'outils	Performances correctes
Fonctions intégrées	WebSockets, JWT, Swagger natifs	Nécessite bibliothèques tierces	Très complet (ORM, admin)

6.2.4 Base de données : PostgreSQL

PostgreSQL est choisi pour sa robustesse, son support ACID et ses fonctionnalités avancées.



FIGURE 6.4 – PostgreSQL

6.3 Implémentation

6.3.1 Organisation du backend (NestJS)

Le backend est organisé en plus de 25 modules, chacun contenant :

- **Controllers** : Exposent les API REST.
- **Services** : Logique métier.
- **Entities** : Modèles TypeORM liés à la base de données.
- **DTOs** : Objets de transfert de données avec validation.

Exemple de module : RepairsModule

Listing 6.1 – Structure du module Repair

```
// repairs.module.ts
@Module({
  imports: [TypeOrmModule.forFeature([Repair, HistoryRepair, Device, Customer])],
  controllers: [RepairController],
  providers: [RepairService, HistoryRepairService],
  exports: [RepairService]
})
export class RepairsModule {}
```

6.3.2 Organisation du frontend (React)

Le frontend est organisé par fonctionnalités avec plus de 45 pages :

- **pages/** : Pages principales (Login, Dashboard, Stock, Réparations, Administration).
- **components/** : Composants réutilisables (DynamicTable, FormModal, etc.).
- **store/** : Gestion d'état Redux Toolkit (authSlice, repairsSlice, stockSlice, etc.).
- **services/** : API service centralisé avec Axios.
- **assets/** : Images, logos.

Exemple de store Redux

Listing 6.2 – Slice Redux pour l'authentification

```
const authSlice = createSlice({
  name: 'auth',
  initialState: { user: null, isAuthenticated: false, roles: [] },
  reducers: {
    loginSuccess: (state, action) => {
      state.user = action.payload.user;
      state.isAuthenticated = true;
      state.roles = action.payload.roles;
    },
  },
});
```

```
logout: (state) => {  
  state.user = null;  
  state.isAuthenticated = false;  
  state.roles = [];  
}  
}  
});
```

6.3.3 Gestion des rôles et navigation dynamique

Un utilisateur peut avoir plusieurs rôles simultanément. Le menu de navigation est généré dynamiquement en fonction des rôles de l'utilisateur connecté. Chaque rôle donne accès à un ensemble spécifique de pages :

- **Administrateur** : Toutes les pages + pages d'administration.
- **Coordinateur** : Affectation, contrôle qualité, transfert.
- **Technicien** : Réparations affectées, diagnostic.
- **Réceptionniste** : Clients, appareils, réparations, factures.
- **Gestionnaire de stock** : Stock, transferts, approbations.

6.3.4 Seed de la base de données

Le projet inclut un script `seed.bat` qui insère les données initiales nécessaires au fonctionnement :

- **type_model** : Types de modèles (smartphone, écouteurs, etc.).
- **level_repair** : Niveaux de réparation avec prix.
- **all_part** : Liste complète des pièces détachées.
- **repair_action** : Actions de réparation possibles.
- **expertise_reason** : Raisons de mise hors garantie.
- **notes_customer** : Notes types pour le client.
- **customer_request** : Demandes clients standards.
- **list_fault** : Problèmes courants.
- **accessory** : Accessoires.

Un script séparé (`seed-user.js`) utilise `argon2` pour hacher le mot de passe de l'administrateur initial.

6.4 Scénarios d'utilisation

6.4.1 Authentification

La page de login permet aux utilisateurs de s'authentifier avec leurs identifiants.

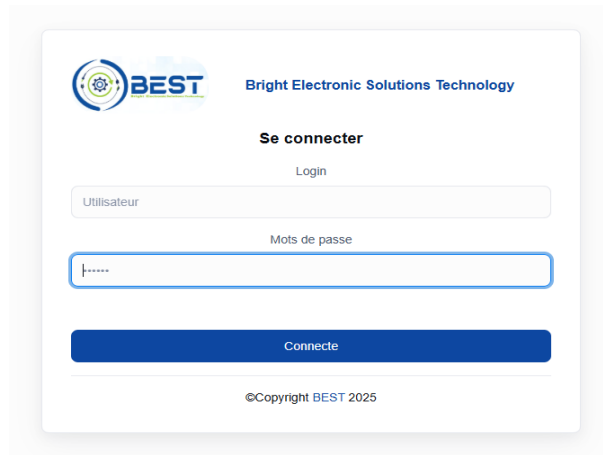


FIGURE 6.5 – Page d'authentification

6.4.2 Tableau de bord

Le tableau de bord s'adapte dynamiquement aux rôles de l'utilisateur :

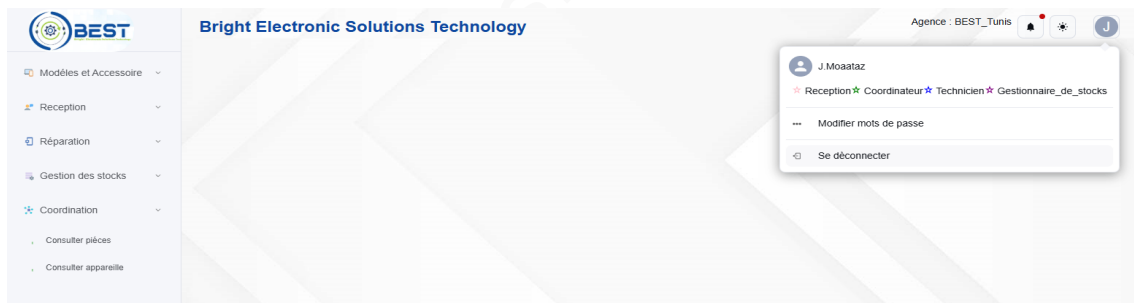


FIGURE 6.6 – Tableau de bord avec quatre rôles

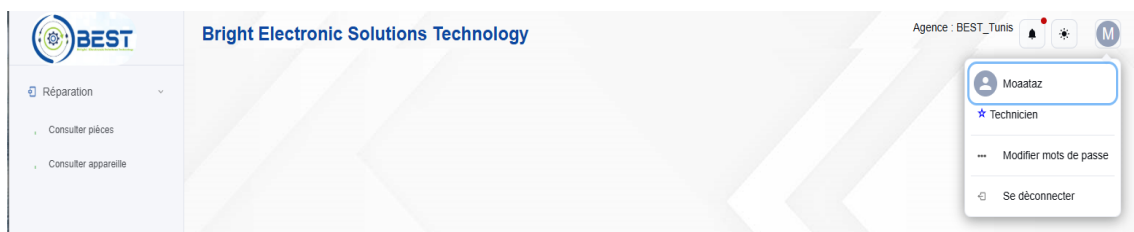


FIGURE 6.7 – Tableau de bord avec un seul rôle

6.4.3 Gestion des agences

L'administrateur peut ajouter, modifier et supprimer des agences.

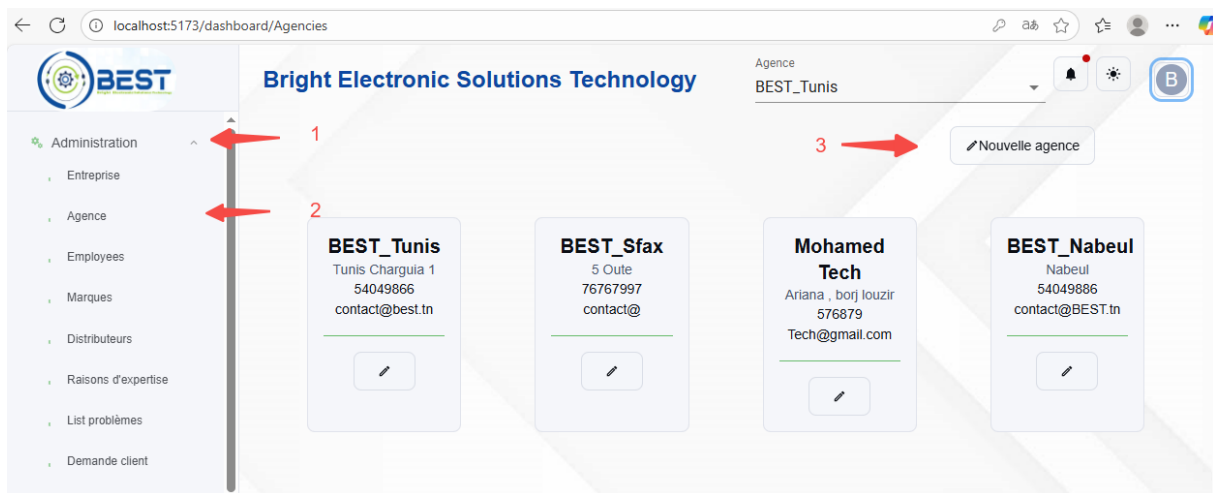


FIGURE 6.8 – Navigation pour ajouter une agence

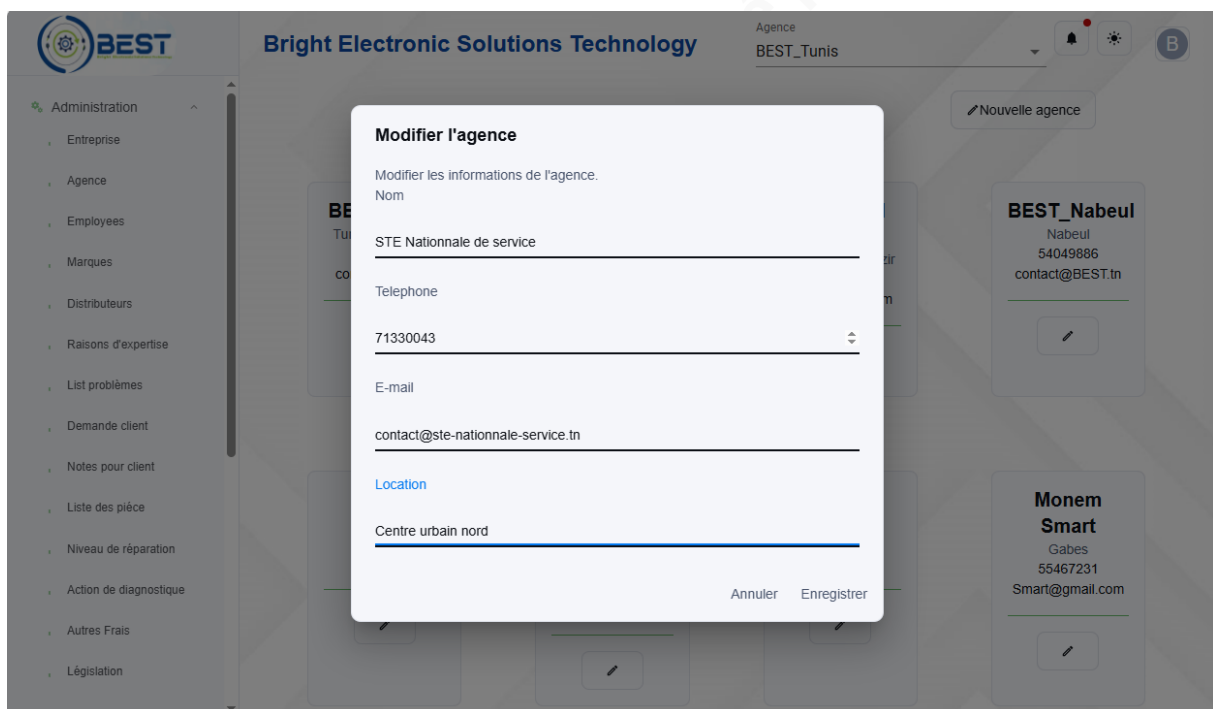


FIGURE 6.9 – Formulaire d'ajout d'une agence

6.4.4 Gestion des utilisateurs

L'administrateur ajoute des utilisateurs en leur attribuant un ou plusieurs rôles.

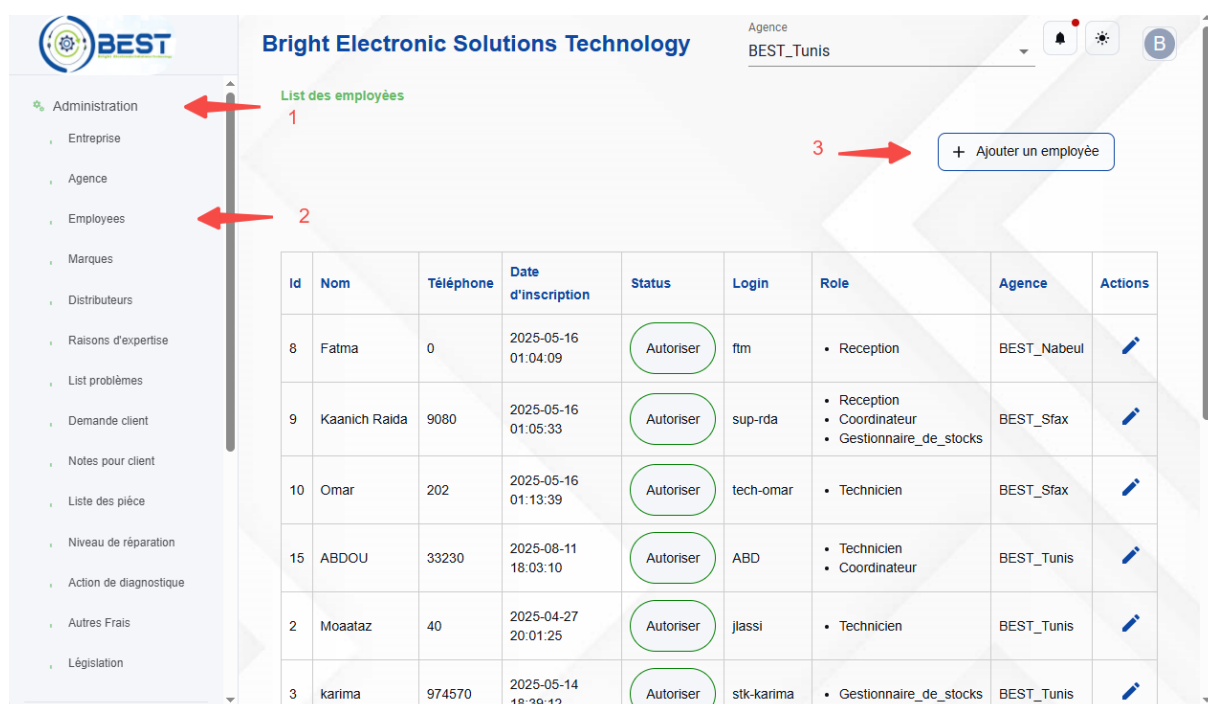


FIGURE 6.10 – Navigation pour ajouter un utilisateur

Ajouter un nouvelle employée

Nom: _____

Téléphone: _____

54049686

Login: _____

yasmin

Mots de passe: _____

1234

Status: ☒ Autoriser ☐ Bloqué

Role:
☐ Reception ☐ Coordinateur ☐ Technicien ☐ Gestionnaire_de_stocks

Agence:
 STE Nationale de service

Annuler Confirmer

FIGURE 6.11 – Formulaire d'ajout d'un utilisateur avec sélection de rôles

6.4.5 Gestion des marques et modèles

L'administrateur ajoute les marques, et le réceptionniste ajoute les modèles.

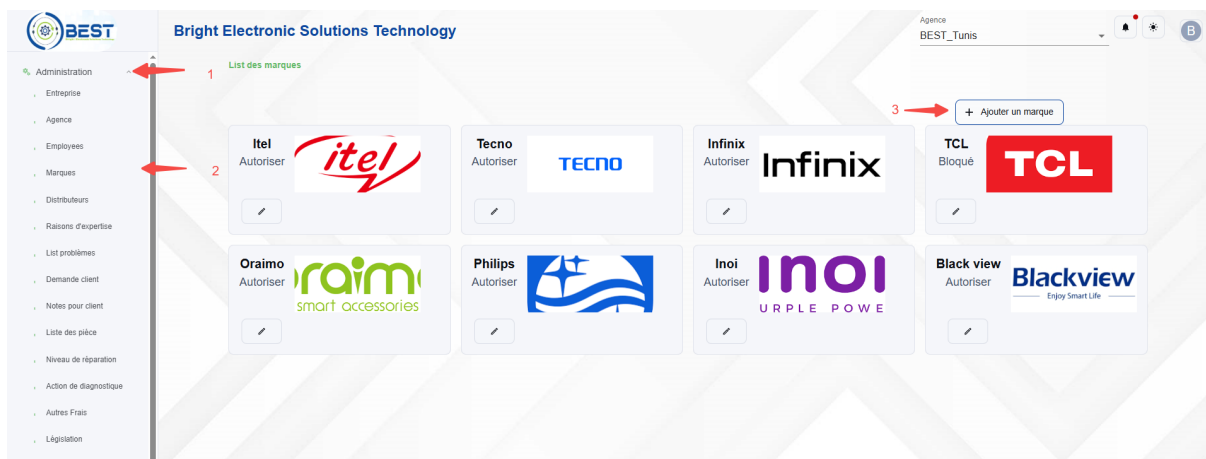


FIGURE 6.12 – Navigation pour ajouter une marque

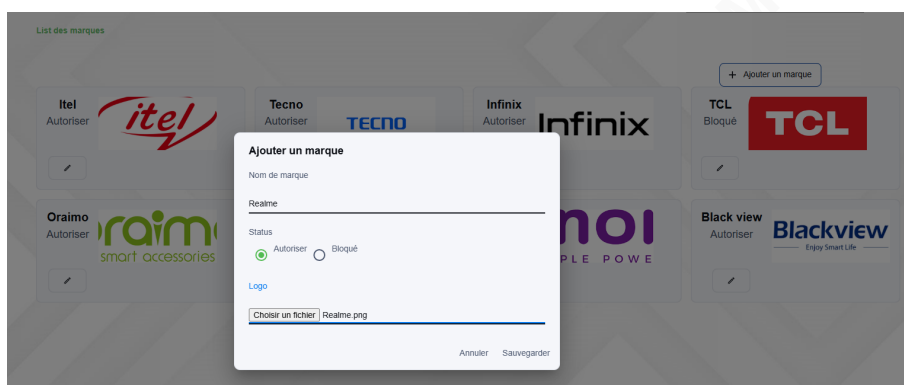


FIGURE 6.13 – Formulaire d'ajout d'une marque

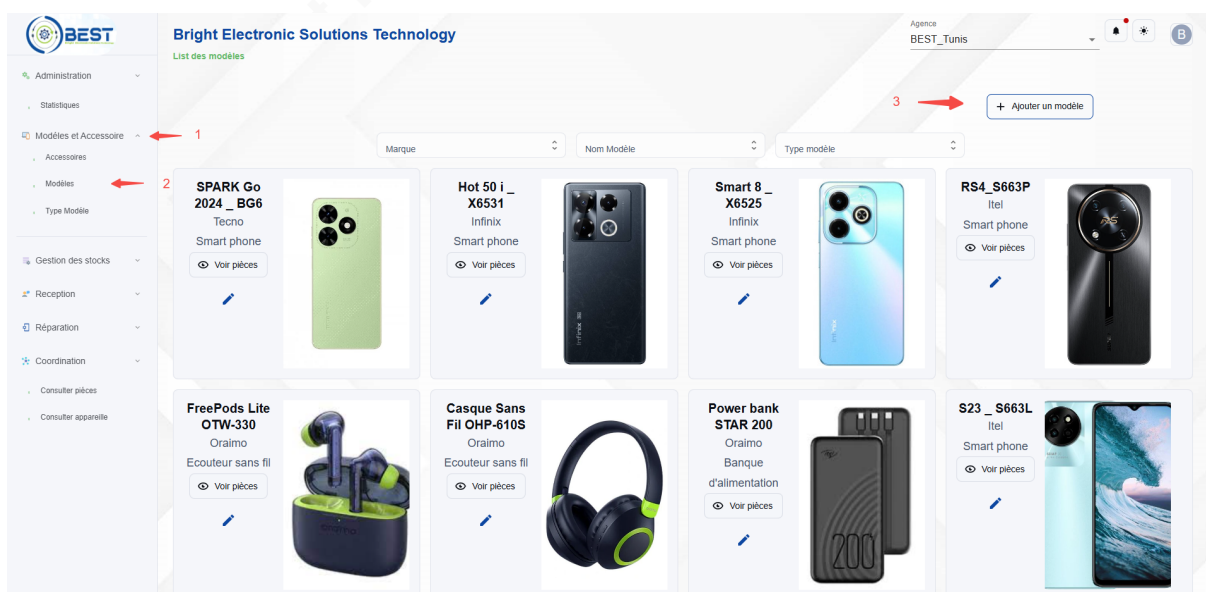


FIGURE 6.14 – Navigation pour ajouter un modèle

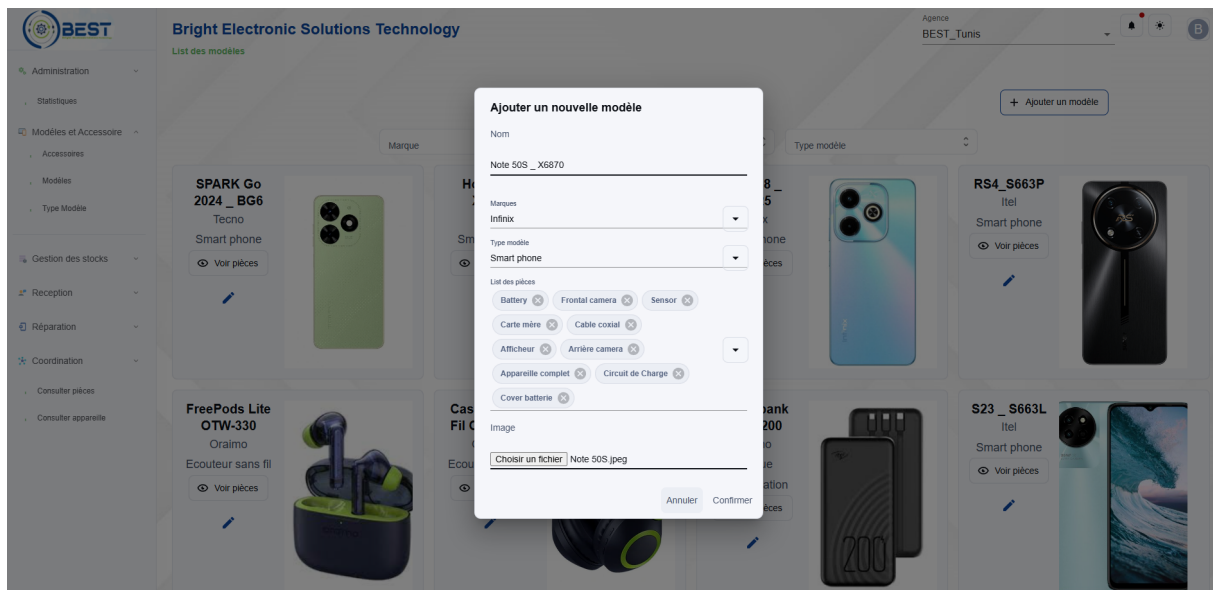


FIGURE 6.15 – Formulaire d'ajout d'un modèle

6.4.6 Réception d'un produit

La réception d'un produit se fait en trois étapes :

1. Ajout ou recherche du client.
2. Enregistrement de l'appareil (numéro de série, modèle).
3. Saisie des détails de réparation (problèmes, accessoires, garantie).

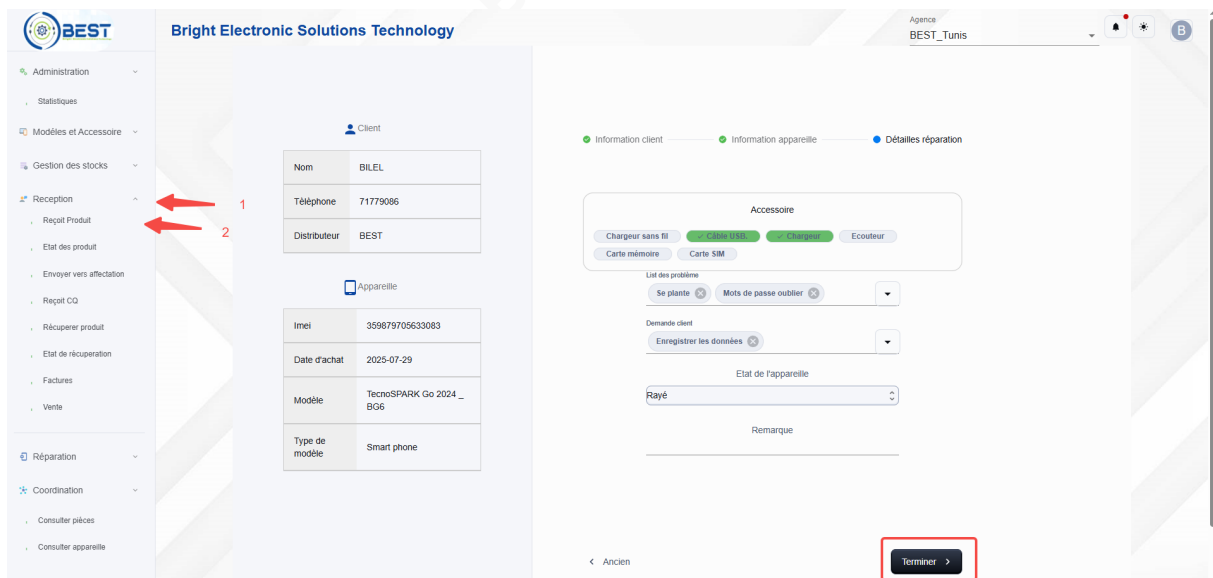


FIGURE 6.16 – Réception d'un produit – navigation et formulaire

Après cette insertion, un fichier PDF contenant toutes les informations est généré. Ce document se compose de deux parties : l'une remise au client (preuve de dépôt) et l'autre conservée avec l'appareil pour le suivi.



Cliant: BILEL	Code Appareil: 300	État appareil: Rayé
Téléphone: 71779086	Marque: Tecno	Accessoires: Chargeur • Câble USB.
Distributeur: BEST	Modèle: SPARK Go 2024 _ BG6	
Reçu le: 21/08/2025	Type: Smart phone	
Reçu par: B.Hammouda Bilel	Imei: 359879705633083	
Agence: BEST_Tunis	Date achat: 29/07/2025	
Problèmes: Mots de passe oublier • Se plante	Remarque: Aucune remarque	Demande client: Enregistrer les données

Législations Applicables

- Les appareils datant de plus de 90 jours après réparation deviennent la propriété de l'utilisateur.
- La garantie ne couvre pas les dommages causés par l'utilisateur.
- Les appareils réparés hors service agréé par la marque pendant la période de garantie seront directement hors garantie.

BEST
Adresse : Rue Ezzi, Nabeul.
Banque: BIAT Chargeur (84)
RIB: 6410612575
MF: 1877406/A/AM000

Agence: BEST_Tunis
Téléphone: 54048606
E-mail: contact@best.tn
E-Adresse: Tunis Chargeur 1



Cliant: BILEL	Code Appareil: 300	État appareil: Rayé
Téléphone: 71779086	Marque: Tecno	Accessoires: Chargeur • Câble USB.
Distributeur: BEST	Modèle: SPARK Go 2024 _ BG6	
Reçu le: 21/08/2025	Type: Smart phone	
Reçu par: B.Hammouda Bilel	Imei: 359879705633083	
Agence: BEST_Tunis	Date achat: 29/07/2025	
Problèmes: Mots de passe oublier • Se plante	Remarque: Aucune remarque	Demande client: Enregistrer les données

Document généré le 21/08/2025

FIGURE 6.17 – PDF généré pour la réparation

6.4.7 Affectation au technicien

Le réceptionniste envoie le dossier au coordinateur pour affectation.



FIGURE 6.18 – Envoi du dossier vers l'affectation

Le coordinateur valide les produits dans sa liste d'affectation puis les assigne à un technicien.



FIGURE 6.19 – Validation des produits en attente d'affectation

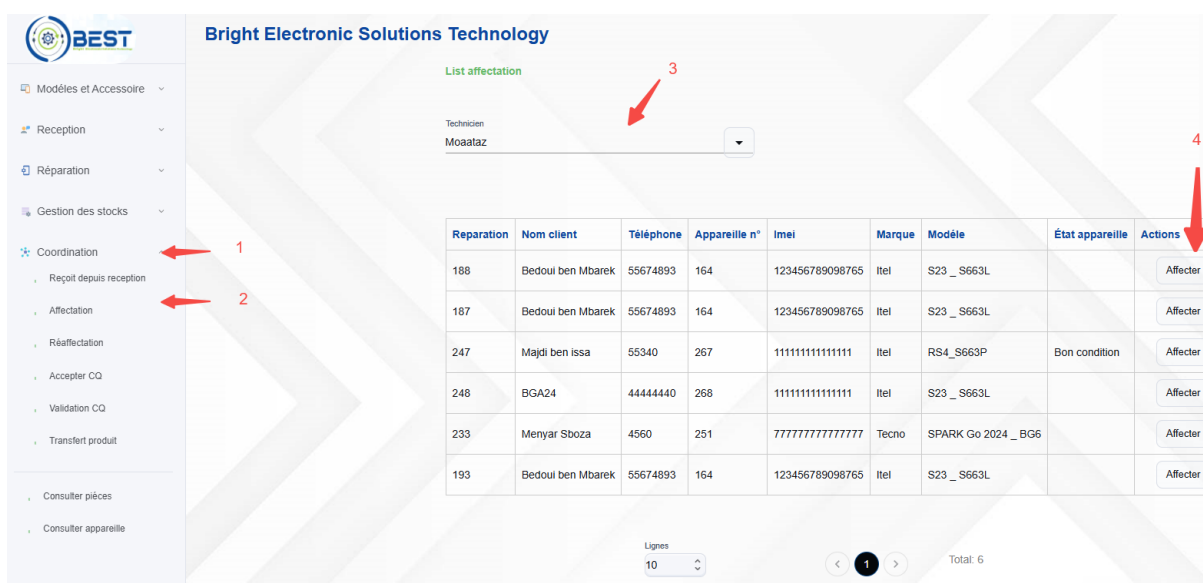


FIGURE 6.20 – Affectation d'un produit à un technicien

6.4.8 Réparation par le technicien

Le technicien accepte l'affectation, réalise le diagnostic et saisit les détails dans le système.

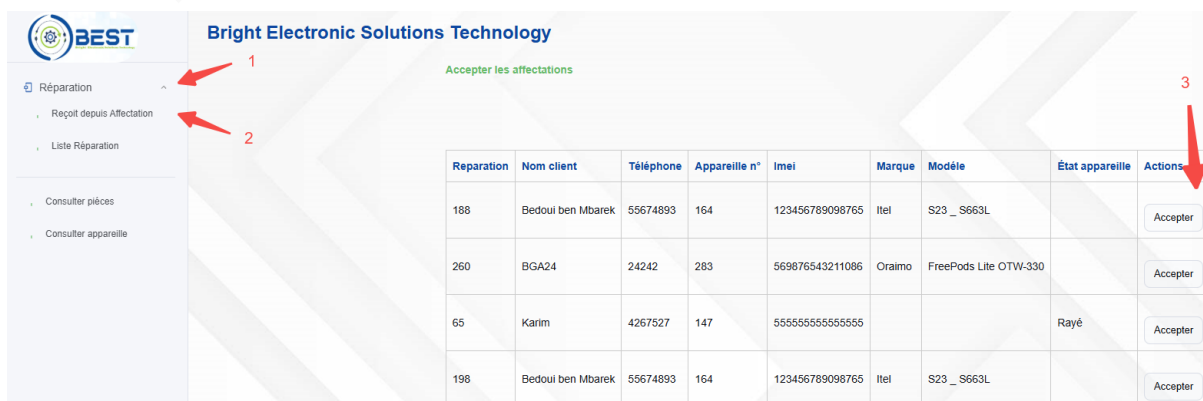


FIGURE 6.21 – Le technicien accepte les affectations

FIGURE 6.22 – Saisie des détails de réparation

6.4.9 Contrôle qualité

Le coordinateur contrôle l'état du produit (physiquement et dans le système), puis approuve ou refuse.

- **Approbation** : La réparation est validée, un PDF récapitulatif est généré.
- **Refus** : La réparation est retournée au technicien pour correction.

FIGURE 6.23 – Contrôle qualité

6.4.10 Récupération par le client

Le réceptionniste enregistre la récupération du produit dans le système. Il peut sélectionner plusieurs produits si le même client récupère plusieurs appareils.

BEST Bright Electronic Solutions Technology

Agence : BEST_Tunis

Modèles et Accessoires

- Reception
 - Reçut Produit
 - Etat des produit
 - Envoyer vers affectation
 - Reçut CQ
 - Récupérer produit
 - Etat de récupération
 - Factures
 - Vente
- Réparation

Client

Nom client Téléphone Distributeur Remarque

0 Distributeur

Récupérer

Reparation	Nom client	Téléphone	Appareille n°	Imei	Marque	Modèle	État appareil	Dernier état
☐	262	Kamel Maaoulya	54087987	285	667754894456740	Oraimo	Power bank STAR 200	Prêt à récupérer

Lignes 10

Total: 1

FIGURE 6.24 – Récupération des produits par le client

6.5 Conclusion

Ce chapitre de réalisation a permis de concrétiser les choix méthodologiques et techniques effectués lors de la modélisation. Il présente les technologies employées, l'architecture mise en place ainsi que les scénarios d'utilisation développés pour répondre aux besoins. Cette phase a permis de traduire les spécifications en solutions opérationnelles, avec une interface ergonomique, un fonctionnement fiable et une structure modulaire.

Chapitre 7

Tests et validation

7.1 Introduction

La qualité du logiciel est une préoccupation centrale dans le développement de BEST Management System. Ce chapitre présente les stratégies de test mises en œuvre, les outils utilisés et les résultats obtenus pour garantir la fiabilité et la stabilité du système.

7.2 Stratégie de test

Trois niveaux de test ont été implémentés :

1. **Tests unitaires (frontend)** : Validation des composants React et des reducers Redux.
2. **Tests d'intégration (backend)** : Validation des services NestJS et des endpoints API.
3. **Tests E2E (Playwright)** : Validation des parcours utilisateur complets.

7.3 Tests unitaires frontend

7.3.1 Outil utilisé : Vitest + Testing Library

Le frontend utilise Vitest 4.1.8 avec happy-dom comme environnement de test, complété par @testing-library/react et @testing-library/jest-dom.

7.3.2 Résultats

- **70 suites de test**
- **678 tests individuels**
- **Couverture** : Composants clés (Login, Dashboard, Tableaux, Formulaires).

7.3.3 Exemple de test

Listing 7.1 – Test unitaire d'un composant

```
import { render, screen } from '@testing-library/react';
import { LoginPage } from './LoginPage';

describe('LoginPage', () => {
  it('renders login form', () => {
    render(<LoginPage />);
    expect(screen.getByLabelText(/login/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /connecter/i })).
      toBeInTheDocument();
  });
});
```

7.4 Tests d'intégration backend

7.4.1 Outil utilisé : Jest + Supertest

Le backend utilise Jest avec Supertest pour tester les endpoints API et les services NestJS.

7.4.2 Résultats

- 78 suites de test
- 801 tests individuels
- **Couverture** : Tous les modules (Authentification, Utilisateurs, Réparations, Stock, Transferts, etc.).

7.4.3 Exemple de test d'intégration

Listing 7.2 – Test d'intégration d'un endpoint API

```
import { Test } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import * as request from 'supertest';
import { AppModule } from './app.module';

describe('AuthController (e2e)', () => {
  let app: INestApplication;
```

```

beforeAll(async () => {
  const module = await Test.createTestingModule({
    imports: [AppModule],
  }).compile();
  app = module.createNestApplication();
  await app.init();
});

it('POST /auth/login should authenticate valid user', () => {
  return request(app.getHttpServer())
    .post('/auth/login')
    .send({ login: 'admin', password: 'admin123' })
    .expect(201)
    .expect(res => {
      expect(res.body.accessToken).toBeDefined();
    });
});

afterAll(async () => {
  await app.close();
});
});

```

7.5 Tests E2E Playwright

7.5.1 Configuration

Les tests E2E utilisent Playwright avec :

- **baseURL** : `http://localhost:5173`
- **Navigateur** : Chromium uniquement
- **webServer** : `npm run dev` (port 5173)

7.5.2 Scénarios de test

89 tests E2E couvrant les scénarios critiques :

TABLE 7.1 – Répartition des tests E2E par module

Module	Tests	Scénarios couverts
Login	4	Connexion, déconnexion, accès refusé
Dashboard	4	Affichage, navigation, rôles

Agences (CRUD)	4	Création, lecture, modification, suppression
Employés (CRUD)	4	Gestion des employés
Stock (simple)	6	Bins, références, prix
Rôles sidebar	5	Navigation par rôle
Admin CRUD	20	Marques, modèles, distributeurs
Modèles/Access.	3	Gestion des accessoires
Réception	16	Clients, appareils, réparations, PDF
Coord./Réparation	9	Affectation, CQ, suivi
Stock (complexe)	8	Transferts, approbations, alertes
Cas limites	6	Erreurs, validations, cas spéciaux

7.5.3 Exemple de test E2E

Listing 7.3 – Test E2E Playwright pour le login

```
import { test, expect } from '@playwright/test';

test('should login with valid credentials', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[name="login"]', 'admin');
  await page.fill('[name="password"]', 'admin123');
  await page.click('button[type="submit"]');
  await expect(page).toHaveURL(/\/dashboard/);
  await expect(page.locator('text=Tableau de bord')).toBeVisible();
});

test('should reject invalid credentials', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[name="login"]', 'wrong');
  await page.fill('[name="password"]', 'wrong');
  await page.click('button[type="submit"]');
  await expect(page.locator('text=Identifiants invalides')).toBeVisible();
});
```

7.6 Bilan des tests

Type de test	Suites	Tests	Outil
Tests unitaires (frontend)	70	678	Vitest + Testing Library
Tests intégration (backend)	78	801	Jest + Supertest
Tests E2E	—	89	Playwright

TABLE 7.2 – Bilan de la couverture des tests

7.7 Conclusion

La stratégie de test à trois niveaux (unitaire, intégration, E2E) a permis de garantir la fiabilité et la stabilité de l'application. Avec un total de 1568 tests, tous validés, le système bénéficie d'une couverture de test complète couvrant les fonctionnalités critiques et les cas d'usage principaux.

Chapitre 8

Conclusion générale

Ce projet nous a permis de concevoir et de mettre en œuvre une solution de gestion complète pour un service après-vente spécialisé dans la réparation de smartphones. L'application BEST Management System repose sur une architecture moderne et modulaire, garantissant maintenabilité et évolutivité.

Le frontend, développé avec React 19, Vite 6 et Material UI, propose une interface ergonomique, réactive et adaptative. La gestion d'état par Redux Toolkit assure une expérience utilisateur fluide et cohérente.

Le backend, construit avec NestJS et TypeORM, assure la robustesse des services métiers, la sécurité des échanges (JWT, Helmet, Throttler) et une gestion efficace des données. La base de données PostgreSQL garantit un stockage fiable et structuré pour l'ensemble des 30+ entités du système.

Une attention particulière a été portée à l'authentification multi-rôles et à la gestion des droits d'accès, permettant aux utilisateurs d'occuper plusieurs rôles avec un seul compte. Les scénarios d'utilisation réalisés ont confirmé la pertinence des fonctionnalités développées : réception des produits, suivi des réparations, gestion des stocks, transferts, contrôle qualité, facturation et notifications.

La phase de test, avec 1568 tests (unitaires, intégration et E2E), a permis de valider la fiabilité du système et de garantir son bon fonctionnement en conditions réelles.

Ce projet constitue une base solide pour de futures évolutions :

- **Tableau de bord analytique** : Indicateurs de performance en temps réel.
- **Intelligence artificielle** : Assistance au diagnostic des pannes.
- **Application mobile** : Suivi pour les clients et les techniciens terrain.
- **Intégration API** : Connexion aux systèmes des marques partenaires.
- **Internationalisation** : Support multilingue.
- **Automatisation** : Workflows avancés avec notifications push.

En conclusion, ce projet illustre la mise en pratique des concepts techniques et organisationnels acquis durant la formation, tout en apportant une réponse concrète et adaptée aux besoins réels du secteur du service après-vente.

Rapport Projet BEST, réalisé par: Moataz

Annexe A

Annexe : Détails des entités et relations

A.1 Les entités du système

Le système comprend plus de 30 entités interconnectées pour gérer l'ensemble du processus SAV :

TABLE A.1 – Description détaillée des entités

Entité	Colonnes
Accessory	id, name
AllPart	id, description
ApproveStock	id, type, date, state, idPartRepair
Bin	id, name, type, branchId
Branch	id, name, location, phone, email
Brand	id, name, logo, status
Company	id, name, headquartersLocation, taxRegisterNumber, rib, bank, logo, quantityAlertStock
CustomerRequest	id, name
Customer	id, name, phone
Device	id, serialnumber, purchaseDate
Distributeur	id, name, phone, email, location, taxRegisterNumber
ExpertiseReason	id, name
HistoryRepair	id, date, step
HistoryStockPart	id, date, step
Invoice	id, paymentMethod, date, state, totalPrice
Legislation	id, name
LevelRepair	id, name, price
ListFault	id, name
Model	id, name, picture

NotesCustomer	id, name
Notification	id, message, type, description
OtherCost	id, price, name, status
OutputList	id, date, remark
PartsPrice	id, price
Reference	id, materialCode, description
Repair	id, warrenty, approveRepair, newserialnumber, actuel-lybranch, files, partsNeed, remark, deviceStateRe-ceive
RepairAction	id, name
Sale	id, state, totalPrice, date
StockAlert	id
StockPart	id, remark, serialnumber
Tracability	id
Transfert	id, delivredBy, sendingDate, fromBranchId, sendU-serId, receivedDate, toBranchId, receiveUserId, type, state, remark, typePart
TypeModel	id, description
User	id, name, phone, password, createdAt, status, login, role

A.2 Les méthodes par entité

TABLE A.2 – Méthodes disponibles par entité

Entité	Méthodes
Accessory	CRUD
AllPart	CRUD
ApproveStock	CRUD, getByRepairId(), getBySaleId(), getByBranchId(), getByType(), getByState()
Bin	CRUD, getByBranchId(), getName(), getByBranchIdAndType()
Branch	CRUD
Brand	CRUD, getAutoriser()
Company	CRUD
CustomerRequest	CRUD
Customer	CRUD, getName(), getByDistributerId()
Device	CRUD, checkDevice(), deviceHasOpenRepair()
Distributeur	CRUD
ExpertiseReason	CRUD

HistoryRepair	CRUD, getByRepairId()
HistoryStockPart	CRUD, getByStockPartId()
Invoice	CRUD, getByState(), getByRepairId(), getByUserId(), getByBranchId()
Legislation	CRUD
LevelRepair	CRUD
ListFault	CRUD
Model	CRUD, findByBrandAuthorised(), getByTypeModel- Id(), getByBrand()
NotesCustomer	CRUD
OtherCost	CRUD
OutputList	CR, getByCustomerId(), getByBranchId()
PartsPrice	CRU, findPartsPriceByModelAndAllPart()
Reference	CRUD, getCompatibleReferences(), findReferenceBy- MaterialCode()
Repair	CRU, updateWithPartsFiles(), getRepairByDevice(), getByNewSerialNumber(), getByActuallyBranch(), ge- tRepairByUser(), getByUserStep()
RepairAction	CRUD
Sale	CRUD, getByState(), getByUserId(), getByBranchId()
StockPart	CRUD, getByBinId(), filterStockParts(), getByBin- Type(), getStateStock()
Tracability	CRUD, getByHistoryRepairId(), getByHistoryStock- PartId()
Transfert	CRUD, getByState(), getFromBranchId(), getByBran- chId()
TypeModel	CRUD
User	CRUD, getUsersAssign(), getUserByLogin(), getByS- tatus(), getByBranchId(), signIn(), logout(), update- Password()

A.3 Relations entre les classes

A.3.1 Relations du module réparation

- **Repair * – * Accessory** : Une réparation peut être associée à plusieurs acces-
soires reçus avec l'appareil.
- **Repair * – * ListFault** : Une réparation peut contenir plusieurs problèmes identi-
fiés.
- **Repair * – * CustomerRequest** : Une réparation peut inclure plusieurs demandes
du client.

- **Repair * – * NotesCustomer** : Une réparation peut avoir plusieurs notes destinées au client.
- **Repair * – * ExpertiseReason** : Une réparation peut avoir plusieurs raisons d'expertise (hors garantie).
- **Repair * – 1 RepairAction** : Une action de réparation peut être associée à plusieurs réparations.
- **Repair 1 – * OutputList** : Une réparation peut être dans une liste de sortie, qui peut contenir plusieurs réparations.
- **Repair * – 1 Customer** : Un client peut avoir plusieurs réparations.
- **Repair * – 1 Device** : Un appareil peut avoir plusieurs réparations (historique).
- **Repair 1 – * ApproveStock** : Une réparation peut avoir plusieurs approbations de pièces.
- **Repair * – * Transfert** : Une réparation peut être transférée plusieurs fois.
- **Repair 1 – * HistoryRepair** : Une réparation a plusieurs étapes dans son historique.
- **Repair 1 – 1 Invoice** : Une réparation peut avoir une facture (hors garantie).
- **Repair * – 1 User** : Un technicien peut être affecté à plusieurs réparations.

A.3.2 Relations du module client et appareil

- **Customer * – 1 Distributeur** : Chaque client appartient à un distributeur.
- **Device * – 1 Model** : Les appareils sont d'un modèle spécifique.
- **Model * – 1 Brand** : Les modèles appartiennent à une seule marque.
- **Model * – 1 TypeModel** : Un modèle a un seul type (smartphone, écouteurs, etc.).
- **Model * – * AllPart** : Chaque modèle a sa propre liste de pièces compatibles.
- **Model 1 – * PartsPrice** : Les prix des pièces sont définis par modèle.
- **AllPart 1 – * PartsPrice** : Une pièce peut avoir plusieurs prix selon le modèle.
- **PartsPrice * – 1 LevelRepair** : Chaque prix correspond à un niveau de réparation.
- **Brand 1 – * LevelRepair** : Chaque marque a sa propre liste de niveaux.

A.3.3 Relations du module stock

- **ApproveStock * – 1 Sale** : Les approbations peuvent concerner des ventes d'accessoires.
- **ApproveStock 1 – 1 StockPart** : Chaque approbation est liée à une pièce en stock.

- **StockPart * – * Transfert** : Les pièces peuvent être transférées plusieurs fois.
- **StockPart * – 1 Bin** : Chaque pièce est stockée dans une seule case.
- **StockPart * – 1 Reference** : Chaque pièce a une seule référence.
- **StockPart 1 – * HistoryStockPart** : Chaque pièce a un historique de mouvements.
- **OutputList * – 1 Customer** : Les listes de sortie appartiennent à un client.
- **Reference 1 – * Model** : Une référence peut concerner un modèle.
- **Reference 1 – * AllPart** : Une référence correspond à une seule pièce.

A.3.4 Relations du module utilisateur et traçabilité

- **HistoryRepair 1 – * Tracability** : Chaque étape d'historique peut avoir plusieurs traçabilités.
- **Tracability * – 1 HistoryStockPart** : La traçabilité des mouvements de stock.
- **Tracability * – 1 User** : Chaque action tracée est effectuée par un utilisateur.
- **User 1 – * OutputList** : Un utilisateur peut gérer plusieurs listes de sortie.
- **User 1 – * Invoice** : Un utilisateur peut créer plusieurs factures.
- **User 1 – * Sale** : Un utilisateur peut effectuer plusieurs ventes.
- **User * – 1 Branch** : Un utilisateur appartient à une seule agence.
- **Branch 1 – * Bin** : Une agence peut avoir plusieurs cases de stockage.